



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Душан Стефановић

**ПОРЕЂЕЊЕ МОНОЛИТНЕ И МИКРОСЕРВИСНЕ
АРХИТЕКТУРЕ КРОЗ ТЕСТОВЕ ОПТЕРЕЂЕЊА НА AWS-У**

ЗАВРШНИ РАД
- Основне академске студије -

Нови Сад, 2025. година



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :		
Идентификациони број, ИБР :		
Тип документације, ТД :		Монографска документација
Тип записа, ТЗ :		Текстуални штампани материјал
Врста рада, ВР :		Завршни (Bachelor) рад
Аутор, АУ :		Душан Стефановић
Ментор, МН :		др Себастијан Стоја, доцент
Наслов рада, НР :		Поређење монолитне и микросервисне архитектуре кроз тестове оптерећења на AWS-у
Језик публикације, ЈП :		Српски/ ћирилица
Језик извода, ЈИ :		Српски
Земља публиковања, ЗП :		Република Србија
Уже географско подручје, УГП :		Војводина
Година, ГО :		2025
Издавач, ИЗ :		Ауторски репринт
Место и адреса, МА :		Нови Сад, Трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)		6/49/9/1/6/0/9
Научна област, НО :		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :		Примењени софтверски инжењеринг
Предметна одредница/Кључне речи, ПО :		микросервиси, монолитна архитектура, поређење перформанси, тестирање оптерећења, мрежна латенција, AWS, ASP.NET Core
УДК		
Чува се, ЧУ :		У библиотеци Факултета Техничких Наука
Важна напомена, ВН :		
Извод, ИЗ :		Овај рад упоређује перформансе монолитне и микросервисне архитектуре на примеру веб апликације QuizHub имплементиране локално (монолит) и на AWS-у. Истраживање је спроведено имплементацијом исте апликације у обе архитектуре и систематским мерењем перформанси коришћењем K6 load testing алата. Рад пружа емпиријске податке и препоруке за избор архитектуре на основу специфичних захтева пројекта.
Датум прихватања теме, ДП :		
Датум одбране, ДО :		
Чланови комисије, КО :	Председник:	др Немања Недић, доцент
	Члан:	др Бојан Јелачић, доцент
	Члан:	
	Члан, ментор:	др Себастијан Стоја, доцент
		Потпис ментора



KEY WORDS DOCUMENTATION

Accession number, ANO :			
Identification number, INO :			
Document type, DT :	Monographic publication		
Type of record, TR :	Textual printed material		
Contents code, CC :	Bachelor Thesis		
Author, AU :	Dušan Stefanović		
Mentor, MN :	Sebastijan Stoja, PhD, Assistant professor		
Title, TI :	Comparing Monolithic and Microservices Architectures Through Load Testing on AWS		
Language of text, LT :	Serbian		
Language of abstract, LA :	Serbian/English		
Country of publication, CP :	Serbia		
Locality of publication, LP :	Vojvodina		
Publication year, PY :	2025		
Publisher, PB :	Author's reprint		
Publication place, PP :	Faculty of Tehnical Sciences, D. Obradovića 6, 21000 Novi Sad		
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	6/49/9/1/6/0/9		
Scientific field, SF :	Electrical and computer engineering		
Scientific discipline, SD :	Applied Software Engineering		
Subject/Key words, S/KW :	Microservices, Monolithic Architecture, Performance Comparison, Load Testing, Network Latency, AWS, ASP.NET Core		
UC			
Holding data, HD :	Library of the Faculty of Tehnical Sciences, Novi Sad, Serbia		
Note, N :			
Abstract, AB :	This thesis compares the performance of monolithic and microservices architectures using the QuizHub web application implemented locally (monolith) and on AWS. The research was conducted by implementing the same application in both architectures and systematically measuring performance using the K6 load testing tool. The thesis provides empirical data and recommendations for architecture selection based on specific project requirements.		
Accepted by the Scientific Board on, ASB :			
Defended on, DE :			
Defended Board, DB :	President:	Nemanja Nedic, PhD, Assistant professor	
	Member:	Bojan Jelacic, PhD, Assistant professor	
	Member:		
	Member, Mentor:	Sebastijan Stoja, PhD, Assistant professor	Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

Број:

ЗАДАТАК ЗА ЗАВРШНИ РАД

Датум:

Студијски програм:	Примењено софтверско инжењерство		
Студент:	Душан Стефановић	Број индекса:	ПР 40/2020
Степен и врста студија:			
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	др Себастијан Стоја, доцент		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Поређење монолитне и микросервисне архитектуре кроз тестове оптерећења на AWS-у

ТЕКСТ ЗАДАТКА:

Извршити преглед литературе, идентификовати могуће поделе постојећих сервиса на микросервисе користећи методе доступне у литератури и имплементирати поделу на AWS-у. Извршити компаративне анализе перформанси монолитне и микросервисне архитектуре на конкретном примеру веб апликације и упоредити резултати кроз тестове оптерећења.

Руководилац студијског програма:	Ментор рада:
др Селаков Александар, ванредни професор	др Себастијан Стоја, доцент

Примерак за: [?] - Студента; [?] - Ментора



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1. Увод	1
1.1. Мотивација	1
1.2. Циљ рада	1
1.3. Садржај рада	2
1.4. Структура рада	2
2. Теоријски оквир	3
2.1. Монолитна архитектура	3
2.1.1. Карактеристике монолитне архитектуре	3
2.1.2. Предности монолитне архитектуре	3
2.1.3. Недостаци монолитне архитектуре	4
2.2. Микросервисна архитектура	5
2.2.1. Карактеристике микросервисне архитектуре	5
2.2.2. Предности микросервисне архитектуре	5
2.2.3. Недостаци микросервисне архитектуре	6
2.3. Компаративна анализа архитектура	7
2.3.1. Критеријуми за поређење	7
2.3.2. Препоруке за избор архитектуре	7
2.3.3. Хибридни приступ	8
3. Имплементација система QuizHub	9
3.1. Преглед апликације QuizHub	9
3.1.1. Функционалне карактеристике	9
3.1.2. Употребљене технологије	10
3.2. Монолитна архитектура	12
3.2.1. Структура монолитне апликације	13
3.2.2. База података	13
3.2.3. Инсталација и подешавање монолитне апликације	14
3.3. Микросервисна архитектура	15
3.3.1. Структура микросервисне апликације	16
3.3.2. Комуникација између сервиса	16
3.3.3. API Gateway	17
3.3.4. Независне базе података	17
3.3.5. Инсталација и подешавање микросервисне апликације	18
3.4. Frontend апликација	19
3.4.1. Структура frontend апликације	19
3.4.2. Комуникација са backend-ом	19
3.4.3. Управљање стањем апликације	20
3.5. Разлике у имплементацији	20
3.5.1. Архитектурне разлике	20
3.5.2. Перформансне импликације	21
4. Методологија тестирања перформанси	22
4.1. Циљеви тестирања	22
4.2. Алат за тестирање	22
4.3. Тест сценарији	23
• Лако оптерећење	23
• Средње оптерећење	23

•	Тешко оптерећење	23
4.4.	Симулација корисничког понашања	23
4.5.	Метрике перформанси	23
4.6.	Клијентско окружење	24
4.7.	Извршавање тестова	24
4.8.	Анализа и визуализација резултата	25
4.9.	Ограничења методологије	25
5.	Резултати и анализа	26
5.1.	Преглед резултата	26
5.2.	Анализа времена одзива	27
5.2.1.	Поређење p95 перцентила	28
5.3.	Анализа пропусности система	29
5.3.1.	Линеарност скалирања	30
5.4.	Анализа стопе грешака	30
5.4.1.	Анализа узрока грешака	31
5.4.2.	Импликације стопе грешака	32
5.4.3.	Корекција за валидну компарацију	32
5.5.	Факторска анализа разлика у перформансама	32
5.5.1.	Мрежна латенција као доминантан фактор	32
5.5.2.	Додатно оптерећење микросервиса	33
5.5.3.	Поређење под једнаким условима	34
5.6.	Скалабилност архитектура	34
5.6.1.	Понашање под растућим оптерећењем	35
5.6.2.	Потенцијал за хоризонтално скалирање	35
5.7.	Синтеза резултата	36
5.7.1.	Контекстуализација резултата	36
5.7.2.	Препоруке на основу резултата	37
6.	Закључак	38
7.	Литература	40
8.	Биографија	41

Списак Слика

Слика 3.1. Шема монолитне апликације	12
Слика 3.2. Шема микросервисне апликације	15
Слика 5.1. Поређење просечног времена одзива монолитне и микросервисне архитектуре	27
Слика 5.2. Поређење р95 времена одзива монолитне и микросервисне архитектуре	28
Слика 5.3. Поређење пропусности монолитне и микросервисне архитектуре	29
Слика 5.4. Поређење стопе грешака монолитне и микросервисне архитектуре	30

Списак додатних појашњења

Додатак 3.1. Приказ структуре монолитне апликације.....	13
Додатак 3.2. Приказ сервиса монолитне апликације.....	14
Додатак 3.3. Приказ сервиса микросервисне апликације.....	16
Додатак 3.4. Приказ сервиса по портovima	17
Додатак 3.5. Конфигурација микросервиса	18
Додатак 3.6. Приказ структуре фронтенд апликације.....	19
Додатак 3.7. Базни URL код монолитне архитектуре	19
Додатак 3.8. Базни URL код микросервисне архитектуре	19
Додатак 4.1. Пример команде за покретање теста	24

Списак табела

Tabela 5.1. Преглед резултата перформанс тестирања.....	26
---	----

Листа скраћеница

ACID - Atomicity, Consistency, Isolation, Durability

API - Application Programming Interface

ASP.NET - Active Server Pages .NET

AWS - Amazon Web Services

CD - Continuous Deployment

CI - Continuous Integration

CLI - Command Line Interface

CORS - Cross-Origin Resource Sharing

CSS - Cascading Style Sheets

DOM - Document Object Model

EC2 - Elastic Compute Cloud

ECR - Elastic Container Registry

gRPC - gRPC Remote Procedure Call

HMR - Hot Module Replacement

HTTP - HyperText Transfer Protocol

ID - Identifier

IP - Internet Protocol

JSON - JavaScript Object Notation

JWT - JSON Web Token

LINQ - Language Integrated Query

ORM - Object-Relational Mapping

RDS - Relational Database Service

REST - Representational State Transfer

RTT - Round Trip Time

SPA - Single Page Application

SQL - Structured Query Language

TCP - Transmission Control Protocol

URL - Uniform Resource Locator

YAML - YAML Ain't Markup Language

1. Увод

Са растом сложености софтверских система и повећањем броја корисника, избор одговарајуће софтверске архитектуре постаје кључан за успех апликације. У свету развоја софтвера постоје два доминантна приступа организацији апликације: монолитна и микросервисна архитектура [1]. Монолитна архитектура представља традиционални приступ у коме је целокупна апликација развијена као јединствена, нераздвојива целина. Насупрот томе, микросервисна архитектура организује апликацију као скуп малих, независних сервиса који међусобно комуницирају преко мрежних протокола [1], [2].

Иако микросервисна архитектура нуди бројне предности попут бољег скалирања, независног развоја сервиса и флексибилности у избору технологија, она са собом носи и одређене изазове. Повећана сложеност инфраструктуре, потреба за управљањем више различитих сервиса, као и мрежна комуникација између сервиса могу утицати на перформансе система.

1.1. Мотивација

Избор између монолитне и микросервисне архитектуре представља један од најважнијих одлука у раним фазама развоја софтвера. Многи развојни тимови се суочавају са дилемом да ли да изаберу једноставност монолитне архитектуре или флексибилност микросервисне архитектуре. Ова одлука често се доноси на основу теоријских разматрања и процена, без конкретних емпиријских података о перформансама у реалном окружењу.

Додатни мотив за истраживање представља чињеница да се већина студија фокусира на велике корпоративне системе, док је релативно мало истраживања посвећено поређењу перформанси ових архитектура на средњим веб апликацијама. Постоји потреба за емпиријском евалуацијом која би пружила конкретне податке о перформансама обе архитектуре у контролисаном окружењу [3].

1.2. Циљ рада

Циљ овог рада је да спроведе детаљну компаративну анализу перформанси монолитне и микросервисне архитектуре на конкретном примеру веб апликације. Кроз имплементацију исте функционалности у оба архитектонска приступа и систематско мерење добијених резултата, посебна пажња посвећена је времену одзива система под различитим оптерећењима, као и пропусности која се изражава кроз број обрађених захтева у јединици времена. Поред тога, анализирана је и поузданост система, мерена преко стопе грешака, као и утицај мрежне комуникације на укупан учинак микросервисног приступа. На основу ових параметара, рад настоји да понуди објективне и емпиријски утемељене метрике које могу помоћи развојним тимовима да донесу информисану одлуку приликом избора одговарајуће архитектуре за сопствене пројекте.

1.3. Садржај рада

У овом раду представљено је поређење перформанси монолитне и микросервисне архитектуре кроз имплементацију и тестирање веб апликације за управљање квизовима *QuizHub*. Апликација је развијена у две верзије: монолитна верзија имплементирана локално и микросервисна верзија имплементирана на AWS (*Amazon Web Services*) облаку.

За потребе истраживања спроведено је детаљно тестирање перформанси коришћењем алата K6, при чему су симулирана три различита сценарија оптерећења система: лако (5 корисника), средње (20 корисника) и тешко (50 корисника). Мерене су кључне метрике перформанси укључујући просечно време одзива, 95-ти перцентил времена одзива, пропусност система и стопу грешака.

1.4. Структура рада

Рад је организован кроз шест главних поглавља која систематски воде кроз теоријске основе, имплементацију и евалуацију система.

У **другом поглављу** представљен је теоријски оквир који обухвата детаљан опис монолитне и микросервисне архитектуре, њихове кључне карактеристике, предности и недостатке. Такође су приказани критеријуми на основу којих се доноси одлука о избору одговарајуће архитектуре.

Треће поглавље описује имплементацију веб апликације *QuizHub* у обе архитектуре. Представљене су коришћене технологије, структура система, као и специфичности имплементације сваке од архитектура. Посебна пажња посвећена је организацији микросервисне архитектуре у три независна сервиса.

У **четвртном поглављу** дефинисана је методологија тестирања перформанси. Описани су тест сценарији, коришћени алати, метрике које су мерене, као и окружење у коме је тестирање спроведено.

Пето поглавље садржи детаљне резултате тестирања перформанси обе архитектуре. Резултати су представљени кроз табеле и графиконе који омогућавају визуелну компарацију перформанси под различитим оптерећењима.

У **шестом поглављу** дата је детаљна анализа и дискусија добијених резултата. Идентификовани су кључни фактори који утичу на перформансе, као и препоруке за практичну примену.

Седмо поглавље представља закључак рада у коме су сумирани главни доприноси истраживања и предложени правци будућег рада.

2. Теоријски оквир

Теоријски оквир поставља основу за разумевање архитектонских приступа који су предмет овог истраживања и представља неопходан контекст за анализу перформанси спроведену у наставку рада. У оквиру овог поглавља најпре се разматрају кључне карактеристике монолитне архитектуре, њене предности и ограничења, након чега се на исти начин анализира микросервисни архитектонски модел. Потом се приступа систематској компарацији два приступа кроз дефинисане критеријуме, као и формулисању препорука за избор оптималне архитектуре у зависности од потреба конкретне система. На крају, посебна пажња посвећена је хибридним приступима који комбинују елементе обе архитектуре, што одражава еволутивну природу савремених софтверских система. Овако постављена структура омогућава свеобухватно разумевање теоријских основа пре уласка у практичну анализу и интерпретацију резултата тестирања.

2.1. Монолитна архитектура

Монолитна архитектура представља традиционални приступ развоју софтверских апликација у коме се целокупна апликација развија, имплементира и извршава као јединствена, недељива целина [4]. У монолитној архитектури, све компоненте апликације - презентациони слој, пословна логика и слој за приступ подацима - интегрисани су у једну извршну јединицу која се имплементира на серверу.

2.1.1. Карактеристике монолитне архитектуре

Монолитна архитектура представља традиционалан приступ развоју софтверских система и одликује се тиме што се целокупна апликација развија као један јединствен пројекат са заједничком базом изворног кода. Сви чланови развојног тима раде у оквиру истог репозиторијума, користећи заједничке ресурсе и паралелно доприносећи истој кодној основи. Пуштање апликације у рад обавља се као један интегрисан процес, што значи да свака измена — чак и она минимална — захтева поновно постављање целокупног система. Такође, апликација се извршава као једна процесна инстанца на серверу, при чему сви њени модули деле исте системске ресурсе, укључујући меморију и процесорско време. Архитектура се ослања и на заједничку базу података којој приступају сви делови система, обично путем унификованих табела и структура података. Комуникација између различитих функционалних целина одвија се синхроно, путем директних позива метода и функција, без потребе за мрежним протоколима или посредничким слојевима.

2.1.2. Предности монолитне архитектуре

Монолитна архитектура, нарочито у контексту мањих пројеката и тимова, пружа низ практичних предности које доприносе једноставности развоја и ефикаснијем управљању процесом. Развој монолитне апликације одвија се у јединственом и добро познатом окружењу, што програмерима омогућава да користе устаљене алате и технологије без потребе за управљањем сложене инфраструктуром или координацијом између више независних сервиса. Оваква архитектура такође поједностављује процес тестирања, јер се све компоненте налазе у оквиру

исте извршне инстанце, па се интеграционо тестирање може изводити без покретања одвојених сервиса или додатних API интеграција.

Посебну предност монолит нуди у раним фазама развоја, када је брзо додавање и проширивање функционалности од великог значаја. Нови модули могу се уграђивати директно у постојећи систем, без потребе за конфигурисањем посебних сервиса или сложених међусистемских интерфејса. Истовремено, оваква организација система захтева мање инфраструктурно оптерећење, јер један сервер или кластер сервера може бити довољан за покретање једне или више идентичних инстанци апликације.

Поред тога, монолитна архитектура омогућава веома брзу локалну комуникацију између компоненти, јер се све интеракције извршавају кроз директне позиве метода у меморији, што је значајно ефикасније од мрежне комуникације која је карактеристична за микросервисне системе. На крају, процес пуштања апликације у рад додатно је поједностављен, будући да се целокупан систем поставља као једна извршна целина, без потребе за сложеним оркестрационим механизмима.

2.1.3. Недостаци монолитне архитектуре

Иако монолитна архитектура нуди бројне предности, њена ограничења постају све израженија како систем расте и постаје комплекснији. Један од највећих изазова односи се на скалирање, јер се монолитна апликација може скалирати само као целина. То значи да, чак и када је потребно повећати капацитет само једног модула, неопходно је скалирати читав систем, што доводи до неефикасног коришћења ресурса и увећава трошкове инфраструктуре. Додатно, монолит подразумева и технолошку зависност, пошто је комплетан систем обично развијен у оквиру истог програмског језика, *framework*-а и пратећих алата. Прелазак на нови технолошки стек или употреба различитих технологија за различите компоненте у пракси је веома сложена и ретко изводљива.

Како апликација временом постаје обимнија, база кода природно расте, што доводи до значајног повећања сложености. Програмерима постаје све теже да разумеју интегрисану структуру и међусобне зависности унутар система, што успорава развој нових функционалности и повећава вероватноћу увођења грешака. У исто време, процеси изградње, тестирања и постављања апликације постају све дужи, јер свака измена, без обзира на то колико је мала, захтева комплетан *build* и *deploy* читавог система. Ова чињеница повећава и ризик приликом пуштања у рад: грешка у једном модулу може довести до пада целе апликације.

Додатни проблем представља и паралелизација развоја. Када више тимова истовремено ради над заједничком базом кода, долази до конфликта, потребе за интензивном координацијом и отежаног независног развоја различитих функционалности. Све ово чини монолитну архитектуру мање погодном за велике и динамичне системе, као и за пројекте са брзом еволуцијом функционалних захтева.

2.2. Микросервисна архитектура

Микросервисна архитектура представља архитектурни приступ у коме се апликација структурира као колекција малих, аутономних сервиса. Сваки сервис је независна компонента која обавља специфичну пословну функцију и комуницира са другим сервисима преко добро дефинисаних API интерфејса, обично коришћењем HTTP(*HyperText Transfer Protocol*)/REST(*Representational State Transfer*) протокола [1], [5].

2.2.1. Карактеристике микросервисне архитектуре

Микросервисна архитектура представља савремени приступ развоју сложених софтверских система и заснива се на принципу разбијања апликације на мање, функционално независне целине. Основна идеја овог модела јесте декомпозиција система по пословним доменима, при чему сваки сервис обрађује јасно дефинисану пословну функцију, у складу са начелима *domain-driven design*-а. На тај начин, сваки микросервис функционише као засебна логичка и техничка целина која може бити развијана, тестирана, пуштана у рад или скалирана независно од осталих сервиса, што значајно повећава флексибилност и убрзава развојни циклус.

Једна од кључних карактеристика овакве архитектуре јесте децентрализовано управљање подацима. Уместо једне централне базе, сваки микросервис поседује сопствену базу података и самостално управља својим моделом података, што омогућава већу независност, избегава међусобну тесну повезаност и умањује ризик од уско грлих архитектонских ограничења. Комуникација између сервиса обавља се путем мрежних протокола, најчешће коришћењем HTTP/REST интерфејса, gRPC-а или система порука. Ова комуникација може бити синхрона или асинхрона, али у сваком случају подразумева мрежну интеракцију, што микросервисима омогућава да буду распоређени, скалирани и извршавани на различитим деловима инфраструктуре.

Додатна предност микросервисне архитектуре је технолошка хетерогеност, јер различити сервиси могу бити имплементирани уз употребу различитих програмских језика, framework-а или технологија, у складу са њиховим специфичним потребама. Због великог броја независних компоненти, овај архитектонски стил природно захтева и висок степен аутоматизације, како у процесима изградње и тестирања, тако и у управљању deploy-ом и континуалним мониторингом. Управо из тог разлога DevOps праксе представљају кључни део успешне примене микросервисне архитектуре.

2.2.2. Предности микросервисне архитектуре

Микросервисна архитектура пружа значајне предности, нарочито када је реч о великим и комплексним системима који захтевају високу флексибилност и поузданост [6]. Једна од најважнијих предности јесте могућност независног скалирања, јер се сваки сервис може прилагођавати оптерећењу без утицаја на остале делове система. На тај начин, ако се појави повећана потреба за радом само једног сервиса, он се може скалирати изоловано, што омогућава ефикасније и економичније коришћење инфраструктуре.

Додатну вредност доноси технолошка флексибилност, јер развојни тимови имају слободу да бирају најпогодније програмске језике, оквири и базе података за сваки појединачни сервис. Таква хетерогеност омогућава да један сервис буде развијен у Python-у са PostgreSQL базом, док

други може користити *Node.js* и *MongoDB*, прилагођавајући техничка решења специфичним потребама домена.

Микросервиси такође омогућавају ефективну паралелизацију развојног процеса. Различити тимови могу истовремено радити на различитим сервисима без међусобног ометања, што убрзава развој и омогућава сваком тиму потпуну контролу над својом компонентом. Ова архитектура пружа и високу изолацију грешака, јер неисправност у једном сервису не мора утицати на функционисање остатка система. Коришћењем шаблона као што су *circuit breaker* и *fallback* механизми, могуће је задржати стабилност целине чак и у случају делимичног пада појединих сервиса.

Захваљујући оваквом моделу, време од настанка идеје до испоруке функционалности кориснику значајно је скраћено. Мањи сервиси омогућавају брже развијање, тестирање и пуштање у рад, па организације могу чешће уводити промене у *production* окружење. Поред тога, микросервисна архитектура омогућава прецизну алокацију ресурса, јер се сваки сервис може конфигурисати независно, уместо да се ресурси додељују читавом систему као у случају монолита.

2.2.3. Недостаци микросервисне архитектуре

Иако микросервисна архитектура доноси бројне предности, она истовремено уводи и значајан ниво додатне сложености која се највише огледа у управљању инфраструктуром и самим процесима развоја [2], [4]. Системи изграђени на овом приступу често обухватају десетине, па чак и стотине независних сервиса, што захтева употребу напредних инфраструктурних решења, као што су оркестрациони алати типа *Kubernetes*-а, механизми за *service discovery* и обимни мониторинг системи. Оваква организација неминовно ствара комплексније окружење у поређењу са монолитним приступом.

Поред инфраструктурне захтевности, микросервисни системи се суочавају и са проблемом мрежне латенције. Пошто се комуникација између сервиса одвија преко мреже, свака интеракција подразумева додатно кашњење у односу на директне позиве метода својствене монолитима, што може значајно утицати на перформансе система. Ова сложеност се даље одражава и на процес пуштања у рад, јер подизање и координација великог броја сервиса захтевају напредне *CI/CD pipeline*-ове, контејнеризацију и пажљиво управљање верзијама, посебно у ситуацијама када различити сервиси напредују различитом динамиком.

Тестирање микросервисних система такође представља велики изазов. За разлику од монолитне архитектуре, где се све компоненте налазе у једној целини, интеграционо и *end-to-end* тестирање у микросервисном окружењу подразумева подизање више независних сервиса и координацију њихових међусобних интеракција, што додатно повећава комплексност процеса. Дистрибуирана природа система доноси и специфичне проблеме карактеристичне за дистрибуиране системе, укључујући евентуалну конзистентност података, сложено управљање трансакцијама и потребу за механизмима који омогућавају дистрибуирано праћење захтева.

Коначно, микросервисна архитектура често резултира већом потрошњом ресурса. Сваки сервис захтева сопствене меморијске и процесорске ресурсе, па у околностима где је број сервиса велики, укупни трошкови инфраструктуре могу надмашити оне који би били потребни за еквивалентан монолитни систем.

2.3. Компаративна анализа архитектура

Компаративна анализа архитектура има за циљ да продубљено сагледа разлике између монолитног и микросервисног архитектонског приступа, полазећи од претходно представљене теоријске основе. С обзиром на то да избор архитектуре значајно утиче на перформансе, скалабилност, одрживост и дугорочну еволуцију софтверског система, неопходно је анализирати ове моделе кроз јасно постављене критеријуме који омогућавају објективно поређење. У наставку се најпре дефинишу кључни параметри који се користе при евалуацији архитектонских решења, затим се формулишу препоруке за избор најпогоднијег приступа у зависности од потреба конкретне апликације, а на крају се разматра и хибридни модел као практично и често најисплативије решење у реалним условима развоја. На овај начин успоставља се свеобухватан оквир за разумевање архитектонских опција пре анализе њихових перформанси у практичном делу истраживања.

2.3.1. Критеријуми за поређење

Приликом избора одговарајуће архитектуре за софтверски систем неопходно је пажљиво размотрити више међусобно повезаних техничких, пословних и организационих критеријума. Један од кључних фактора јесу перформансе система, које обухватају време одзива апликације, пропусност, латенцију у комуникацији између компоненти и укупну ефикасност коришћења расположивих ресурса. Ове метрике одређују колико брзо и поуздано систем може да одговори на захтеве корисника и да обради велики број истовремених операција.

Скалабилност представља још један важан аспект, јер систем мора бити способан да се прилагоди растућем броју корисника или података. Овде се разматрају могућности хоризонталног скалирања, кроз додавање нових инстанци, као и вертикалног скалирања, путем повећања ресурса постојећих инстанци. Успешан архитектонски избор треба да обезбеди да систем може да расте без значајног смањења перформанси [7].

Поузданост и отпорност на грешке такође су од пресудног значаја. Архитектура мора омогућити да апликација настави са радом и у ситуацијама када поједине компоненте откажу, као и да обезбеди брз и ефикасан опоравак након пада [4]. Поред тога, потребно је проценити и сложеност развоја и одржавања, односно колико времена и ресурса је потребно за увођење нових функционалности, исправљање грешака и одржавање постојеће кодне базе.

Важан критеријум представља и *time-to-market*, односно брзина којом се нове функционалности могу развијати и испоручивати корисницима. Брже испоруке омогућавају организацијама да брже реагују на захтеве тржишта и корисника. На избор архитектуре значајно утиче и величина развојног тима, као и његова организација, број чланова, степен искуства и расподела одговорности. Коначно, потребно је ускладити архитектурни приступ са величином и природом саме апликације, сложеностју пословних захтева, очекиваним бројем корисника и пројектованим растом система.

2.3.2. Препоруке за избор архитектуре

На основу кључних карактеристика и ограничења монолитне и микросервисне архитектуре, могуће је формулисати јасне смернице које могу послужити као основа за избор одговарајућег архитектонског приступа [2]. Монолитна архитектура се обично препоручује у случајевима када

је апликација мања или средње величине и не одликује се високом сложености, као и када развојни тим броји релативно мали број програмера који раде у оквиру централизоване организације. Оваква архитектура је погодна у ситуацијама где су захтеви система стабилни и добро дефинисани, без очекиваних честих промена функционалности. Предност јој даје и чињеница да омогућава брз развој у раним фазама пројекта, што је посебно важно у иницијалним циклусима развоја. Монолит се показује као одговарајуће решење и у околностима у којима није неопходно коришћење различитих технологија за различите делове апликације, где је оптерећење система углавном уједначено, као и када су инфраструктурни ресурси ограничени.

С друге стране, микросервисна архитектура представља бољи избор за велике и сложене апликације са бројним међусобно повезаним функционалностима, посебно када на развоју ради већи број програмера или више тимова који функционишу паралелно. Предност овог приступа огледа се у могућности независног скалирања различитих компоненти система, што је важно у сценаријима где поједини делови апликације имају различите захтеве по питању оптерећења или перформанси. Ова архитектура је посебно погодна када различите функционалне целине имају различите техничке потребе, јер омогућава употребу разнородних технологија у оквиру једног система. Микросервиси су такође препоручљиви у окружењима која захтевају честе испоруке нових функционалности и континуални *deploy*, као и у случајевима када је потребно градити систем отпоран на грешке, који може наставити са радом чак и уколико поједине компоненте откажу. Оваква архитектура најбоље долази до изражаја у организацијама које располажу зрелом *DevOps* културом и одговарајућом инфраструктуром која може подржати сложеност великог броја независних сервиса.

2.3.3. Хибридни приступ

У савременој софтверској индустрији, уобичајена је примена хибридног приступа који комбинује елементе монолитне и микросервисне архитектуре, прилагођавајући се специфичним потребама пројекта. Један од таквих модела јесте модуларни монолит, односно монолитна апликација која је унутрашње организована у јасно дефинисане модуле са лабавим међусобним везама. Иако апликација формално остаје монолит, ова структурна разграничења омогућавају лакшу будућу транзицију у микросервисну архитектуру уколико се за то укаже потреба.

Други чест приступ подразумева постепену миграцију, где се развој иницијално заснива на монолитној архитектури, а како систем временом расте и добија сложености функционалности, поједине критичне компоненте се издвајају у засебне микросервисе. На овај начин систем задржава предности монолита у раним фазама развоја, док се у каснијим фазама уводи флексибилност карактеристична за микросервисни модел. Поред тога, неке организације примењују и селективну употребу микросервиса, где се монолитна архитектура задржава као основа, али се одређене функционалности — попут обраде плаћања или система нотификација — издвајају и реализују као независни сервиси.

Овакви хибридни модели омогућавају да се постигне оптималан баланс између једноставности коју нуди монолитна архитектура и флексибилности, скалабилности и отпорности на грешке које доноси микросервисни приступ. На тај начин систем се може развијати еволутивно, у складу са конкретним техничким и пословним захтевима пројекта.

3. Имплементација система *QuizHub*

У овом поглављу представљена је детаљна имплементација веб апликације *QuizHub* у обе архитектуре - монолитној и микросервисној. Апликација је развијена са циљем да омогући креирање, управљање и извршавање квизова, као и праћење резултата корисника.

3.1. Преглед апликације *QuizHub*

QuizHub је веб апликација за управљање квизовима која омогућава корисницима да креирају, деле и решавају квизове из различитих области знања. Апликација подржава различите типове питања, категоризацију квизова, систем бодовања и праћење најбољих резултата.

3.1.1. Функционалне карактеристике

Апликација обухвата више кључних функционалности које омогућавају свеобухватно управљање квиз системом. Пре свега, систем подржава потпуно управљање корисницима, укључујући регистрацију и аутентификацију, при чему се корисницима додељују различите улоге као што су *Administrator*, *Creator* и *User*. Механизам аутентификације заснован је на употреби JWT (*JSON Web Token*) токена, што омогућава сигурно управљање приступом и сесијама.

Поред управљања корисницима, система подржава и напредно управљање квизовима. Корисници са улогом *Creator* могу креирати квизове који садрже произвољан број питања, дефинисати категорије, постављати временска ограничења и одређивати број бодова за појединачна питања. На овај начин омогућено је креирање разноврсних и структурално различитих квизова прилагођених различитим тематикама и нивоима тежине.

Извршавање квизова представља још један значајан сегмент система. Корисници могу прегледати доступне квизове, одабрати онај који желе да решавају и приступити одговарању на питања у унапред дефинисаном временском оквиру. По завршетку квиза, систем приказује остварен резултат, чиме се кориснику обезбеђује непосредна повратна информација о успешности.

Квизови су организовани по категоријама, као што су спорт, историја или наука, што знатно олакшава претрагу и филтрирање садржаја. Поред тога, систем евидентира све резултате корисника и омогућава преглед њихових постигнућа, као и рангирање на основу остварене успешности. На тај начин корисници могу пратити сопствени напредак, а систем може пружити додатне вредности у виду статистике и конкурентности.

3.1.2. Употребљене технологије

За имплементацију апликације коришћене су следеће технологије:

- **Backend технологије:**

- *ASP.NET Core 8.0* – савремени, високо перформантан Microsoft-ов *framework* за развој веб API-ја и серверских апликација. Подржава cross-platform извршавање, минимални API модел, уграђене механизме за *dependency injection* и одличну интеграцију са *cloud* сервисима, што га чини погодним за скалабилне и продукционе системе.
- *Entity Framework Core 8.0* – ORM (Object-Relational Mapper) који омогућава рад са базом података на нивоу објеката, без потребе за писањем SQL упита. Подржава LINQ, миграције шеме базе и оптимизације перформанси као што су *lazy loading* и *connection pooling*.
- *PostgreSQL 15* – напредна, open-source релациона база података позната по поузданости, стабилности и подршци за комплексне upite, ACID трансакције и напредне типове података. Погодна је за скалабилне производне системе.
- *JWT Bearer Authentication* – механизам за аутентификацију и ауторизацију заснован на токенима. Обезбеђује сигуран приступ API-ју без потребе за стањем на серверу, што је посебно важно за SPA апликације и микросервисне системе.
- *Serilog* – модуларни *framework* за структурисано логовање који подржава записивање догађаја у датотеке, конзолу, *cloud* сервисе и аналитичке платформе. Омогућава прецизно праћење догађаја, што олакшава дијагностику и мониторинг.

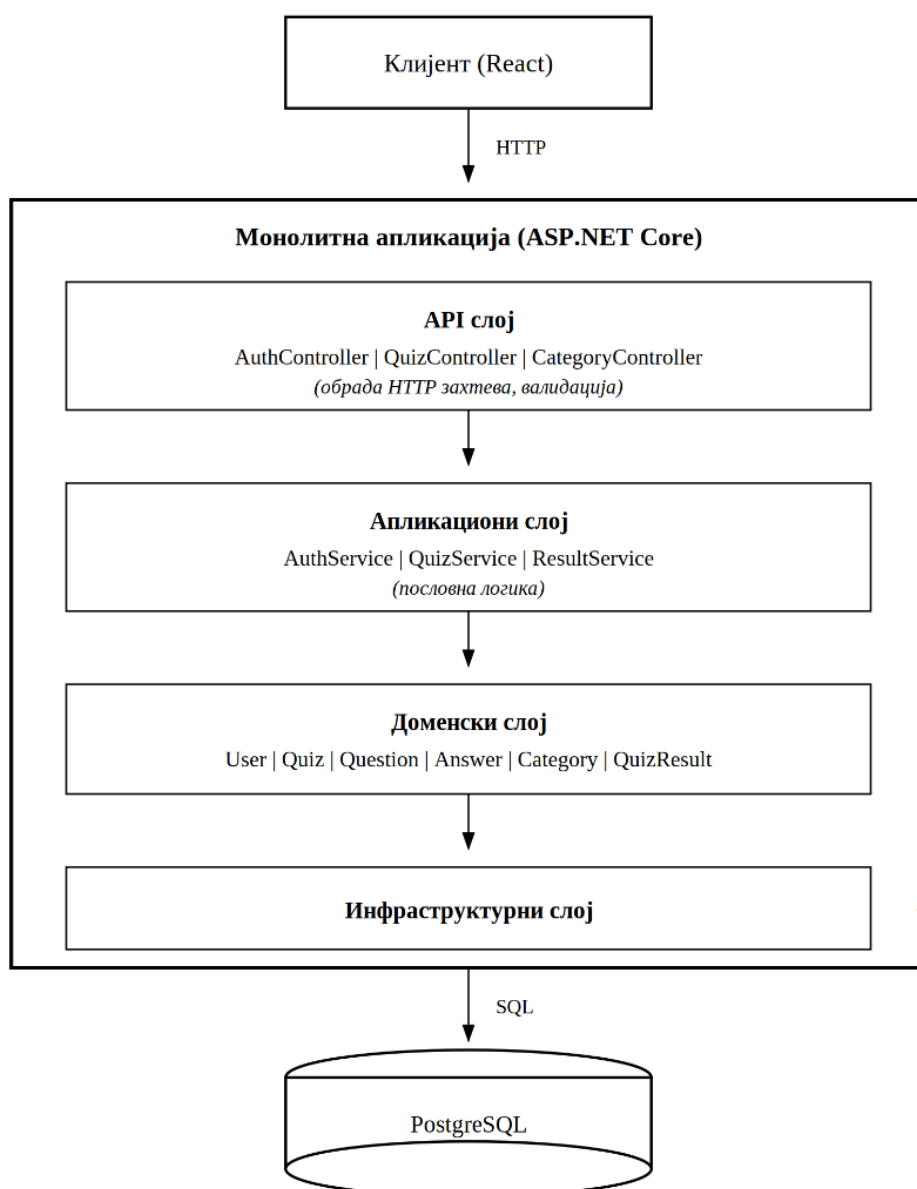
- **Frontend технологије:**

- *React 18.2* – једна од најпопуларнијих библиотека за изградњу корисничког интерфејса. Подржава компонентни приступ, виртуелни DOM и оптимизације перформанси као што су *concurrent rendering* и *automatic batching*.
- *TypeScript 5.2* – проширење JavaScript-а која уводи статичке типове, чиме се повећава поузданост кода и смањује број грешака током развоја. Посебно је користан у већим апликацијама са сложеним моделима података.
- *Vite 5.0* – ултра брз развојни сервер и *build* алат који користи *native ES modules*. Нуди тренутно освежавање (HMR), брзо стартовање пројекта и оптимизован *production build*.
- *Tailwind CSS 3.3* – *utility-first* CSS оквир који омогућава брзу и флексибилну изградњу интерфејса користећи класе мале грануларности. Уклања потребу за ручним писањем CSS-а у већини сценарија.
- *React Router 6.20* – библиотека за навигацију у SPA апликацијама која омогућава дефинисање рута, заштићених страница и динамичког рутирања.
- *Zustand 4.4* – лагана и једноставна библиотека за управљање стањем. Заснива се на минималном API-ју, високој перформантности и једноставној интеграцији са React компонентама.

- *Axios 1.6* – HTTP клијент који поједностављује комуникацију са API-јем, подржава *interceptore*, обраду грешака, конфигурисање *header*-а и отказивање захтева.
- **Инфраструктурне технологије:**
 - *Docker* – платформа за контејнеризацију која изолује апликацију и њене зависности у лагане, преносиве контејнере. Омогућава једноставно постављање, скалирање и реплицирање окружења.
 - *Docker Compose* – алат за оркестрацију вишеконтјнерских апликација. Кроз једну *YAML* датотеку омогућава дефинисање целокупне инфраструктуре (*backend*, *frontend*, база података) и њихово заједничко покретање.
 - *Nginx* – високо перформантан веб сервер и *reverse proxy*. Користи се у микросервисној архитектури за рутирање саобраћаја, *load balancing* и кеширање, што побољшава перформансе и стабилност система.
 - *AWS EC2* – *cloud* сервис за креирање и управљање виртуелним серверима. Омогућава скалабилно и поуздано окружење за постављање микросервиса, са контролом над мрежом, системом и ресурсима.
 - *AWS RDS* – управљана услуга за базе података која аутоматски обавља *backup*, *monitoring*, *scaling* и безбедносне конфигурације. Користи се за *PostgreSQL* у микросервисној имплементацији, чиме се растерећује ручно управљање базом.

3.2. Монолитна архитектура

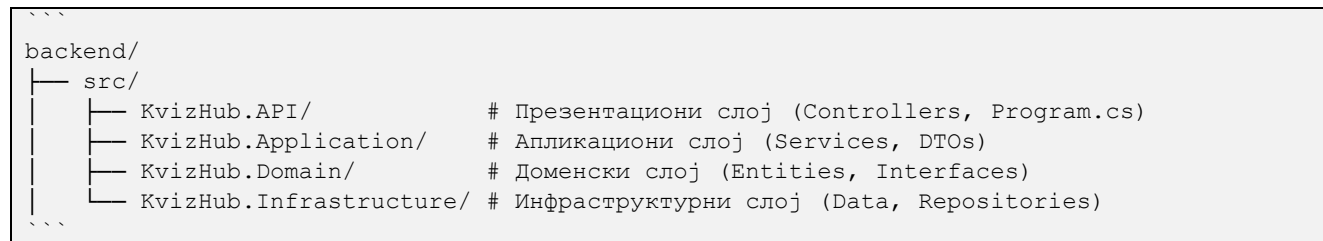
Да би се јасније представила унутрашња организација апликације у монолитној архитектури, приказана је њена структурна шема која илуструје начин на који све компоненте функционишу као јединствена целина. Ова визуелна репрезентација помаже да се сагледа како слојеви система међусобно комуницирају, како се обрађују захтеви корисника и на који начин се приступа заједничкој бази података. Шема додатно наглашава једну од кључних карактеристика монолита – да све функционалности, логика и ресурси раде у оквиру исте апликационе инстанце, без потребе за мрежном комуникацијом између модула.



Слика 3.1. Шема монолитне апликације

3.2.1. Структура монолитне апликације

Монолитна верзија апликације организована је као јединствена *ASP.NET Core* апликација која обједињује све функционалности у једном пројекту. Структура пројекта следи принципе чисте архитектуре (енгл. Clean Architecture) са јасном поделом на слојеве [8]:



Додатак 3.1. Приказ структуре монолитне апликације

KvizHub.API - Овај слој садржи API контролере који обрађују HTTP захтеве и враћају одговоре. Главни контролери су *AuthController* (аутентификација), *QuizController* (управљање квизовима), *CategoryController* (категорије) и *AdminController* (администрација).

KvizHub.Application - Апликациони слој садржи сервисе који имплементирају пословну логику. Овде су дефинисани сервиси као *AuthService*, *QuizService*, *ResultService* који енкапсулирају пословна правила и координирају рад са репозиторијумима.

KvizHub.Domain - Доменски слој садржи доменске ентитете (*User*, *Quiz*, *Question*, *Answer*, *Category*, *QuizResult*) и интерфејсе репозиторијума. Овај слој не зависи од других слојева.

KvizHub.Infrastructure - Инфраструктурни слој садржи имплементацију репозиторијума, *ApplicationDbContext* за *Entity Framework Core*, и скрипте за иницијализацију базе података.

3.2.2. База података

Монолитна апликација користи једну *PostgreSQL* базу података која садржи све табеле потребне за рад апликације. Главне табеле су:

- **AspNetUsers** - корисници система (укључујући *Identity framework* табеле)
- **Categories** - категорије квизова
- **Quizzes** - основни подаци о квизовима
- **Questions** - питања унутар квизова
- **Answers** - понуђени одговори за питања
- **QuizResults** - резултати корисника

Све табеле су повезане релационим везама (енгл. *foreign key constraints*), што обезбеђује интегритет података. *Entity Framework Core* управља миграцијама и креирањем шеме базе података.

3.2.3. Инсталација и подешавање монолитне апликације

Монолитна апликација инсталирана је локално коришћењем *Docker Compose*-а. Дефинисана су три сервиса:

```
```yaml
services:
 postgres: # PostgreSQL база података
 backend: # ASP.NET Core API (порт 5000)
 frontend: # React апликација (порт 3050)
```
```

Додатак 3.2. Приказ сервиса монолитне апликације

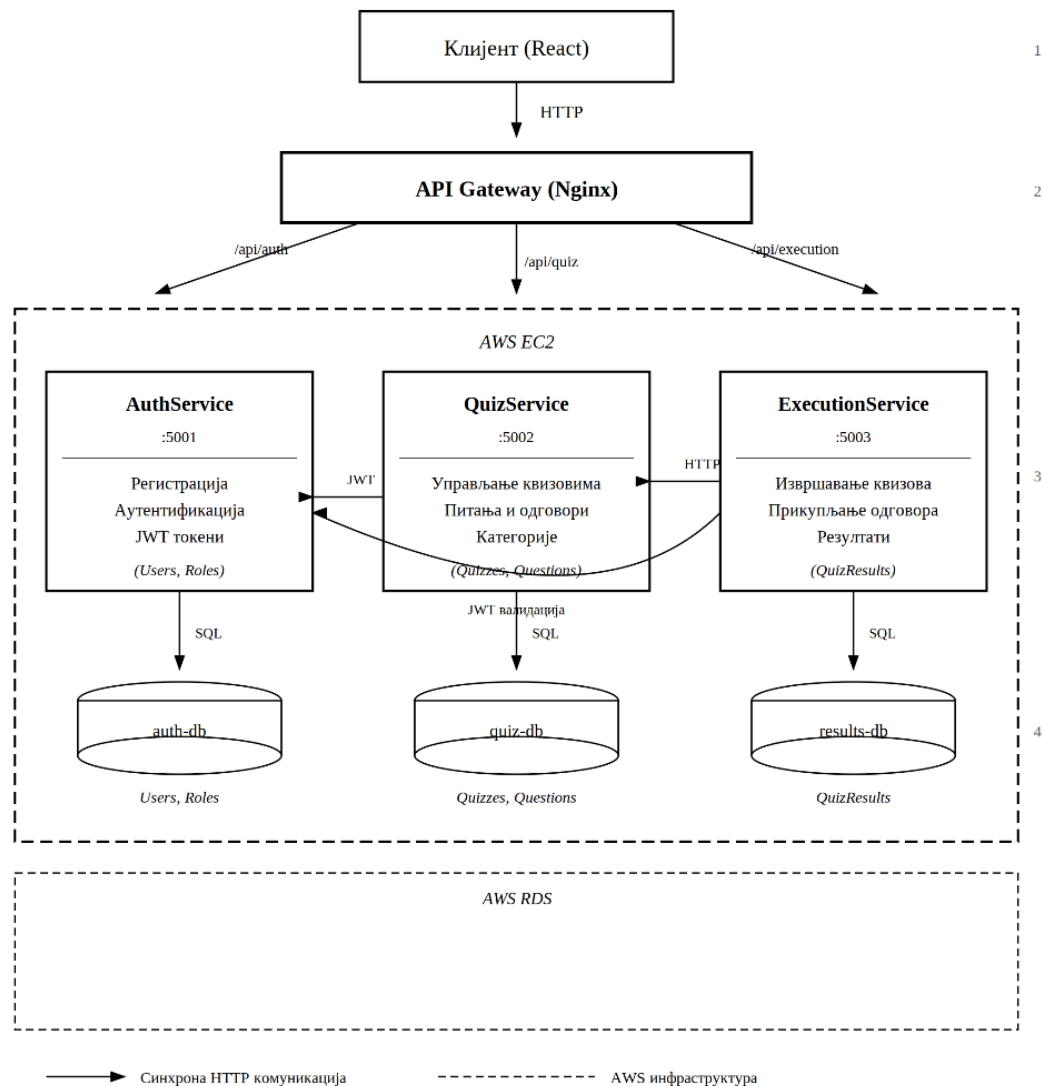
Сви сервиси комуницирају преко приватне *Docker* мреже *kvizhub-network*. *Backend* апликација комуницира са *PostgreSQL* базом података користећи *connection string*, а *frontend* комуницира са *backend*-ом преко HTTP протокола на *http://localhost:5000*.

Процес инсталирања је једноставан:

1. Покретање *'docker-compose up -d'* команде
2. Docker креира *backend* и *frontend* слике (енгл. Images)
3. Контејнери се покрећу и међусобно се повезују
4. База података се иницијализује са основним подацима

3.3. Микросервисна архитектура

Да би се јасно приказала организациона структура система у микросервисној архитектури, представљена је шема која илуструје начин на који су појединачни сервиси раздвојени и међусобно повезани. Ова визуелна целина показује ток захтева од клијента, преко API Gateway-а, до сваког сервиса који независно обрађује свој део логике и комуницира са сопственом базом података. Приказ наглашава кључне принципе микросервисног приступа — децентрализацију, независно извршавање, мрежну комуникацију и јасну разграниченост одговорности — што омогућава већу флексибилност и скалабилност система у односу на монолитно решење.



Слика 3.2. Шема микросервисне апликације

3.3.1. Структура микросервисне апликације

Микросервисна верзија апликације декомпонована је на три независна сервиса, где сваки сервис одговара специфичаном пословном домену [9]:

```

    backend/src/
    ├── KvizHub.AuthService/      # Сервис за аутентификацију
    ├── KvizHub.QuizService/      # Сервис за управљање квизовима
    └── KvizHub.ExecutionService/  # Сервис за извршавање квизова
  
```

Додатак 3.3. Приказ сервиса микросервисне апликације

AuthService (Authentication Service) - Одговоран је за управљање корисницима, регистрацију, пријављивање и издавање JWT токена. Овај сервис има своју базу података која садржи табелу корисника и *Identity framework* табеле. Комуницира на порту 5001.

QuizService (Quiz Management Service) - Управља квизовима, питањима, одговорима и категоријама. Има засебну базу података са табелама *Quizzes*, *Questions*, *Answers* и *Categories*. За ауторизацију валидира JWT токене издате од *AuthService*-а. Комуницира на порту 5002.

ExecutionService (Quiz Execution Service) - Одговоран је за покретање квизова, прикупљање одговора корисника и чување резултата. Има своју базу података са табелом *QuizResults*. Комуницира са *QuizService*-ом да преузме податке о квизу и питањима, и са *AuthService*-ом за валидацију корисника. Комуницира на порту 5003.

3.3.2. Комуникација између сервиса

Микросервиси комуницирају међусобно коришћењем синхроних HTTP/REST позива. Сваки сервис излаже HTTP API који други сервиси могу да позову.

Пример комуникације при извршавању квиза:

1. Корисник шаље захтев *frontend*-у да покрене квиз
2. *Frontend* шаље HTTP POST захтев до *ExecutionService*-а
3. *ExecutionService* валидира JWT токен (може позвати *AuthService*)
4. *ExecutionService* позива *QuizService* да преузме податке о квизу
5. *ExecutionService* креира сесију за извршавање квиза
6. Корисник одговара на питања
7. *ExecutionService* израчунава резултат и чува га у својој бази
8. *Frontend* приказује резултат кориснику

Ова синхрона комуникација уводи мрежну латенцију јер сваки HTTP захтев пролази кроз мрежу, што је један од кључних фактора који утичу на перформансе микросервисне архитектуре [3].

3.3.3. API Gateway

За управљање рутирањем захтева користи се *Nginx* као *API Gateway* [1]. *Nginx* је конфигурисан да рутира захтеве на основу URL путање до одговарајућег микросервиса:

```
```\n/api/auth/**          → AuthService (port 5001)\n/api/category/**      → QuizService (port 5002)\n/api/quiz/**          → QuizService (port 5002)\n/api/execution/**     → ExecutionService (port 5003)\n/api/results/**       → ExecutionService (port 5003)\n```\n
```

Додатак 3.4. Приказ сервиса по портovima

*API Gateway* обезбеђује јединствену тачку уласка за *frontend* апликацију и скрива сложеност микросервисне архитектуре од клијента. Такође обрађује CORS (енгл. *Cross-Origin Resource Sharing*) захтеве и може имплементирати ограничење протока (енгл. *rate limiting*) и распоређивање оптерећења (енгл. *load balancing*).

### 3.3.4. Независне базе података

Сваки микросервис има своју засебну базу података, што је један од кључних принципа микросервисне архитектуре [1], [2]. У имплементираној верзији на AWS-у, коришћене су следеће базе:

- **auth-db** - *PostgreSQL* база за *AuthService* (корисници и улоге)
- **quiz-db** - *PostgreSQL* база за *QuizService* (квизови, питања, категорије)
- **results-db** - *PostgreSQL* база за *ExecutionService* (резултати корисника)

Све три базе података имплементиране су као засебне AWS RDS инстанце. Овај приступ обезбеђује потпуну независност сервиса, али отежава имплементацију трансакција које би обухватале више сервиса [6].

### 3.3.5. Инсталација и подешавање микросервисне апликације

Микросервисна архитектура инсталирана је на AWS облаку коришћењем следећих компоненти:

**AWS EC2** - Виртуелна машина са *Elastic IP* адресом 44.208.207.182 на којој су инсталирани *Docker* контејнери са микросервисима и *Nginx gateway*-ом.

**AWS RDS** - Три засебне *PostgreSQL RDS* инстанце за сваки микросервис. RDS обезбеђује управљане базе података са аутоматским чувањем и опцијама високе доступности (енгл. *high availability options*).

**Docker Compose** - На EC2 инстанци користи се *Docker Compose* за оркестрацију свих микросервиса. Конфигурација дефинише четири сервиса:

```

```yaml
services:
  api-gateway:      # Nginx reverse proxy (port 80)
  auth-service:     # Authentication (port 5001)
  quiz-service:     # Quiz management (port 5002)
  execution-service: # Quiz execution (port 5003)
```

```

Додатак 3.5. Конфигурација микросервиса

Процес инсталације:

1. Креирање EC2 инстанце и додела Elastic IP адресе
2. Креирање RDS база података за све сервисе
3. Креирање Docker слике за све микросервисе
4. Отпремање слике контејнера (image) у Docker Hub или AWS ECR регистар
5. Конфигурисање променљивих окружења (*connection strings, JWT secret*)
6. Покретање *'docker-compose up -d'* на EC2 инстанци
7. Конфигурисање Nginx-а за рутирање захтева
8. Отварање потребних портова у *AWS Security Group* (80, 443)

*Elastic IP* адреса обезбеђује да се IP адреса не мења након рестартовања EC2 инстанце, што је важно за стабилност система и перформанс тестове.

## 3.4. Frontend апликација

Frontend апликација је идентична за обе архитектуре и комуницира са *backend*-ом искључиво преко HTTP API-ја. Апликација је имплементирана као *Single Page Application* (SPA) коришћењем *React framework*-а.

### 3.4.1. Структура frontend апликације

```

 frontend/src/
 ├── components/ # Компоненте корисничког интерфејса
 ├── pages/ # Странице апликације
 ├── services/ # HTTP клијенти за комуникацију са API-јем
 ├── stores/ # Zustand state management
 ├── types/ # TypeScript type дефиниције
 └── utils/ # Помоћне функције

```

Додатак 3.6. Приказ структуре фронтенд апликације

#### Главне странице апликације:

- *Login / Register* - Аутентификација корисника
- *Dashboard* - Преглед доступних квизова,
- *Quiz Detail* - Детаљи појединачног квиза,
- *Quiz Taking* - Решавање квиза са питањима,
- *Results* - Преглед резултата, и
- *Admin Panel* - Администраторски панел (креирање квизова).

### 3.4.2. Комуникација са backend-ом

Frontend комуницира са backend-ом користећи Axios HTTP клијент. Конфигурација базног URL-а се разликује између монолитне и микросервисне архитектуре:

#### Монолитна архитектура:

```

    ````typescript
    baseUrl: 'http://localhost:5000'
    ````

```

Додатак 3.7. Базни URL код монолитне архитектуре

#### Микросервисна архитектура:

```

    ````typescript
    baseUrl: 'http://44.208.207.182'
    ````

```

Додатак 3.8. Базни URL код микросервисне архитектуре

Axios је конфигурисан са пресретачима (енгл. interceptors) који аутоматски додају JWT токен у *Authorization header* за све захтеве, као и систем за руковање грешкама (енгл. error handling) који обрађује 401 *Unauthorized* и друге грешке.

### 3.4.3. Управљање стањем апликације

Апликација користи *Zustand* за управљање глобалним стањем. Кључна стања/складишта (енгл. Stores) су:

- ***authStore*** - Чува податке о пријављеном кориснику и JWT токен
- ***quizStore*** - Управља листом квизова и тренутно селектованим квизом
- ***themeStore*** - Управља *dark/light* темом

*Zustand* је изабран због једноставности и перформанси у односу на *Redux*, а обезбеђује довољну функционалност за потребе ове апликације.

## 3.5. Разлике у имплементацији

Разлике између монолитне и микросервисне архитектуре не огледају се само у теоријским принципима већ и у конкретном начину на који су обе верзије апликације реализоване у пракси. Иако обе имплементације покривају исту пословну логику и подржавају исте функционалности, њихова унутрашња организација, приступ управљању подацима и механизми комуникације суштински се разликују. Управо ова структурна и техничка различитост доводи до значајних разлика у понашању система у реалним условима рада, што се посебно огледа у перформансама под оптерећењем. У наредним потпоглављима анализирају се најважније архитектурне разлике и њихове непосредне перформансне импликације.

### 3.5.1. Архитектурне разлике

Кључне разлике између монолитне и микросервисне имплементације најјасније се уочавају у начину организације кода, структури пројекта и начину комуникације између функционалних целина. Монолитна верзија система реализована је као један јединствен пројекат у оквиру којег су сви модули међусобно повезани и комуницирају директним позивима метода између различитих слојева. Све функционалности ослањају се на једну заједничку базу података која користи механизме страних кључева ради очувања интегритета података. Инсталациони процес је релативно једноставан, јер се комплетна апликација испоручује као једна *Docker* слика, а пошто се све интеракције између компоненти одвијају унутар истог процеса, комуникација се реализује у меморији, без икакве мрежне латенције.

Супротно томе, микросервисна архитектура подразумева раздвајање система на више независних пројеката, од којих сваки има сопствена подешавања, логичку структуру и животни циклус. Сервиси међусобно комуницирају искључиво путем HTTP API позива који се извршавају преко мреже, што уводи додатну латенцију и сложеније руковање комуникационим протоколима. Уместо једне заједничке базе података, сваки сервис има сопствену базу без директних релационих веза са осталима. Оваква организација условљава сложенији процес постављања система у рад, јер се користи више *Docker* слика уз додатно *Nginx* рутирање које омогућава комуникацију између сервиса. Последица оваквог приступа јесте постојање мрежне комуникације која, за разлику од монолитног решења, уводи мерљиву латенцију у систем.

### 3.5.2. Перформансне импликације

Архитектонске разлике између монолитног и микросервисног приступа имају директан и мерљив утицај на перформансе система. Најзначајнији фактор представља мрежна латенција, јер микросервисна архитектура за сваки међусервисни позив мора да користи комуникацију преко мреже, док се у монолитном систему све интеракције извршавају кроз директне позиве метода у меморији. Ова разлика постаје посебно изражена у сценаријима где се микросервисна апликација извршава у cloud окружењу као што је AWS, док се монолит покреће локално, што додатно повећава кашњење у комуникацији.

Поред мрежне латенције, значајан утицај има и процес серијализације и десеријализације података. У микросервисној архитектури сваки HTTP захтев захтева претварање објеката у JSON формат и њихово поновно реконструисање на страни пријемника, што уводи додатни overhead и успорава обраду. Насупрот томе, монолит директно размењује објекте у меморији, без потребе за било каквом серијализацијом.

Додатна сложеност потиче и из успостављања мрежних конекција. Сваки позив између микросервиса захтева успостављање TCP конекције, што представља још један извор латенције који у монолитном систему не постоји, јер се све компоненте извршавају у оквиру истог процеса. На перформансе утиче и начин приступа бази података: док монолит користи један пул конекција ка заједничкој бази, микросервиси морају одржавати одвојене пулове конекција за сваку сопствену базу, што додатно оптерећује системске ресурсе.

Све наведене разлике биће детаљно анализиране и квантитативно приказане у оквиру поглавља посвећеног резултатима перформанс тестова.

## 4. Методологија тестирања перформанси

У овом поглављу описана је методологија која је коришћена за тестирање и компарацију перформанси монолитне и микросервисне архитектуре апликације *QuizHub*.

### 4.1. Циљеви тестирања

Главни циљ перформанс тестирања јесте да се омогући квантитативна и објективна компарација монолитне и микросервисне архитектуре у условима који што верније симулирају стварно коришћење система. У оквиру овог процеса, посебна пажња усмерена је на мерење времена одзива апликације под различитим нивоима оптерећења, као и на процену пропусности система, односно броја захтева које систем може успешно да обради у јединици времена. Поред тога, тестирање обухвата анализу стопе грешака при расту оптерећења, што омогућава увид у стабилност и поузданост обе архитектуре у стресним условима.

Додатни аспект обухвата евалуацију скалабилности, са циљем да се утврди како се свако архитектонско решење понаша при постепеном повећању броја корисника или интензитета захтева. Коначно, тестирање има за задатак и идентификацију потенцијалних перформансних загушења која могу ограничити ефикасност система, било да се ради о спорој комуникацији, ограничењима базе података или неефикасним деловима логике. На овај начин добија се целовита слика о перформансним карактеристикама оба архитектонска приступа.

### 4.2. Алат за тестирање

За спровођење перформанс тестова коришћен је К6, савремени open-source алат за тестирање оптерећења који је развила компанија Grafana Labs. Избор овог алата оправдан је због више његових значајних карактеристика. Пре свега, К6 се одликује једноставношћу употребе, јер омогућава да се тестови пишу у JavaScript-у, што га чини приступачним и програмерима који нису специјализовани за алате за тестирање. Поред тога, К6 подржава генерисање изузетно великог оптерећења, што омогућава реалистичну симулацију рада система под стресним условима и добијање поузданих и репрезентативних резултата.

Додатна предност алата огледа се у опширном скупу метрика које аутоматски прикупља током извршавања тестова, чиме се омогућава детаљна анализа перформанси система. Захваљујући CLI интеграцији, тестови се лако покрећу директно из командне линије, што поједностављује интеграцију у постојеће DevOps процесе. Поред тога, К6 омогућава извоз резултата у JSON формату, што олакшава даљу аналитичку обраду и визуализацију података. Све ове карактеристике чине К6 изузетно погодним алатом за потребе овог истраживања.



### 4.3. Тест сценарији

Дефинисана су три сценарија оптерећења који симулирају различите услове коришћења. Сви сценарији трају 2 минута и користе константан број виртуелних корисника (енгл. constant-vus executor).

- **Лако оптерећење**

- **Број корисника:** 5 виртуелних корисника
- **Трајање:** 2 минута
- **Сврха:** Мерење базних перформанси без значајног оптерећења

- **Средње оптерећење**

- **Број корисника:** 20 виртуелних корисника
- **Трајање:** 2 минута
- **Сврха:** Симулација типичног оптерећења у редовној употреби

- **Тешко оптерећење**

- **Број корисника:** 50 виртуелних корисника
- **Трајање:** 2 минута
- **Сврха:** Тестирање система под великим оптерећењем

### 4.4. Симулација корисничког понашања

Тест скрипта симулира реалистично понашање корисника који користи апликацију. Сваки виртуелни корисник извршава следећу секвенцу акција у циклусу:

1. **Преглед свих квизова** - GET `/api/quiz`
2. **Преглед категорија** - GET `/api/category`
3. **Преглед детаља квида** - GET `/api/quiz/{id}`
4. **Health check** - GET `/health`

После сваке акције следи пауза од 1 секунде, што симулира природно понашање корисника. Користи се реални ID квида из базе података (`1f645276-d6bc-4901-8774-5d8af1e13396``).

### 4.5. Метрике перформанси

К6 током извршавања тестова аутоматски прикупља више кључних метрика које омогућавају свеобухватну анализу перформанси система. Најважнија међу њима је метрика времена одзива (`http_req_duration`), која обухвата просечно време обраде захтева, медијану, као и вредности 95. и 99. перцентила. Ове статистичке мере пружају детаљан увид у брзину реакције система, при

чему су перцентилне вредности од посебног значаја јер указују на то како систем функционише у најнеповољнијим сценаријима, а не само у типичним, просечним условима.

Поред времена одзива, К6 прати и пропусност система, измерену кроз број обрађених захтева у секунди (`http_reqs_rate`), што омогућава процену капацитета система под растућим оптерећењем. Додатно, алат бележи и стопу грешака (`http_req_failed`), односно проценат неуспешних захтева, што представља важан показатељ поузданости система. Комбинација ових метрика омогућава да се перформансе обе архитектуре анализирају на свеобухватан и прецизан начин, уз нагласак на реално корисничко искуство у различитим условима оптерећења.

## 4.6. Клијентско окружење

К6 тестови извршавани су са исте локалне машине на којој је инсталиран монолит. Ово значи:

- **Монолит:** К6 и апликација на истој машини, минимална мрежна латенција
- **Микросервиси:** К6 локално, апликација на AWS-у, значајна мрежна латенција

Ова разлика је намерна и реалистична - монолитне апликације се често инсталирају на приватним серверима, док микросервиси често користе *cloud* инфраструктуру.

## 4.7. Извршавање тестова

Извршено је укупно 6 тестова (3 сценарија × 2 архитектуре):

1. Монолит – Лако оптерећење
2. Монолит - Средње оптерећење
3. Монолит - Тешко оптерећење
4. Микросервиси - Лако оптерећење
5. Микросервиси - Средње оптерећење
6. Микросервиси - Тешко оптерећење

```

```bash
"C:\Program Files\k6\k6.exe" run test-scenarios.js \
  --env BASE_URL=http://localhost:5000 \
  --env TEST_NAME=monolith \
  --env SCENARIO=heavy_load \
  --out json=results-monolith-heavy_load.json
```

```

*Додатак 4.1. Пример команде за покретање теста*

Резултати сваког теста сачувани су у засебном JSON фајлу за даљу анализу.

## 4.8. Анализа и визуализација резултата

По завршетку тестирања, резултати су обрађени коришћењем Python скрипте (`generate-graphs.py`) која:

1. Учитава све JSON фајлове са резултатима
2. Екстрактује кључне метрике
3. Генерише графичке приказе са српским ћириличним ознакама
4. Израчунава процентуалне разлике између архитектура

У оквиру анализе резултата генерисана су четири графичка приказа који омогућавају директно поређење перформанси монолитне и микросервисне архитектуре. Први график илуструје разлике у просечном времену одзива између два архитектонска решења, док други приказује њихову пропусност, односно број захтева које систем може да обради у јединици времена. Трећи график представља стопу грешака и омогућава процену стабилности система под оптерећењем, а четврти се фокусира на p95 време одзива, које пружа увид у понашање система у најоптерећенијим сценаријима. Ови графички прикази заједно омогућавају свеобухватну и визуелно јасну анализу перформанси разлика између обе архитектуре.

## 4.9. Ограничења методологије

Перформанс тестирање спроведено у оквиру овог рада има одређена ограничења која је важно узети у обзир приликом интерпретације резултата. Најпре, постоје разлике у тестном окружењу, будући да је монолитна апликација тестирана локално, док је микросервисна архитектура извршавана на AWS инфраструктури. Ова разлика природно уводи повећану мрежну латенцију у случају микросервиса, али уједно одражава и реалне сценарије у којима се микросервисне апликације најчешће и покрећу у cloud окружењима.

Поред тога, тестирање је било ограничено на read операције, односно на извршавање GET захтева, док write операције као што су POST, PUT и DELETE нису биле укључене у тест сценарио. Ова одлука омогућила је усредсређивање на чисте перформансне разлике настале искључиво због архитектонских фактора, посебно када се узме у обзир да тестови нису садржали механизме аутентификације корисника, како би се елиминисао додатни overhead и задржала фокусираност на саму архитектуру.

Тестови су трајали по два минута, што је довољно за добијање базног увида у понашање система, али не омогућава идентификацију проблема који би се могли јавити током дужег континуираног оптерећења, као што су *memory leak*-ови или деградација перформанси услед продуженог рада.

Ипак, упркос овим ограничењима, примењена методологија обезбеђује валидно поређење две архитектуре и пружа поуздану основу за извлачење значајних закључака о њиховим перформансним карактеристикама.

## 5. Резултати и анализа

У овом поглављу представљени су резултати перформанс тестирања монолитне и микросервисне архитектуре апликације *QuizHub*. Резултати су анализирани и упоређени кроз четири кључне метрике: време одзива, пропусност система, стопа грешака и перцентилне вредности времена одзива.

### 5.1. Преглед резултата

Извршено је укупно шест тестова - три сценарија оптерећења (лако, средње, тешко) за обе архитектуре. Сваки тест је трајао 2 минута и симулирао је реалистично понашање корисника. У табели 5.1 приказан је преглед кључних резултата за све тестове.

| Архитектура  | Сценарио              | Просечно време одзива (ms) | Пропусност (req/s) | Стопа грешака (%) |
|--------------|-----------------------|----------------------------|--------------------|-------------------|
| Монолит      | Лако (5 корисника)    | 4.61                       | 3.98               | 0.0               |
| Монолит      | Средње (20 корисника) | 6.41                       | 15.91              | 0.0               |
| Монолит      | Тешко (50 корисника)  | 5.44                       | 39.79              | 0.0               |
| Микросервиси | Лако (5 корисника)    | 127.99                     | 3.61               | 25.0              |
| Микросервиси | Средње (20 корисника) | 125.36                     | 14.37              | 25.0              |
| Микросервиси | Тешко (50 корисника)  | 129.56                     | 35.83              | 25.0              |

Tabela 5.1. Преглед резултата перформанс тестирања

## 5.2. Анализа времена одзива

Време одзива представља критичну метрику за корисничко искуство. У контексту веб апликација, време одзива испод 100ms се сматра тренутним, 100-300ms се сматра прихватљивим, док време изнад 1 секунде већ негативно утиче на корисничко искуство.



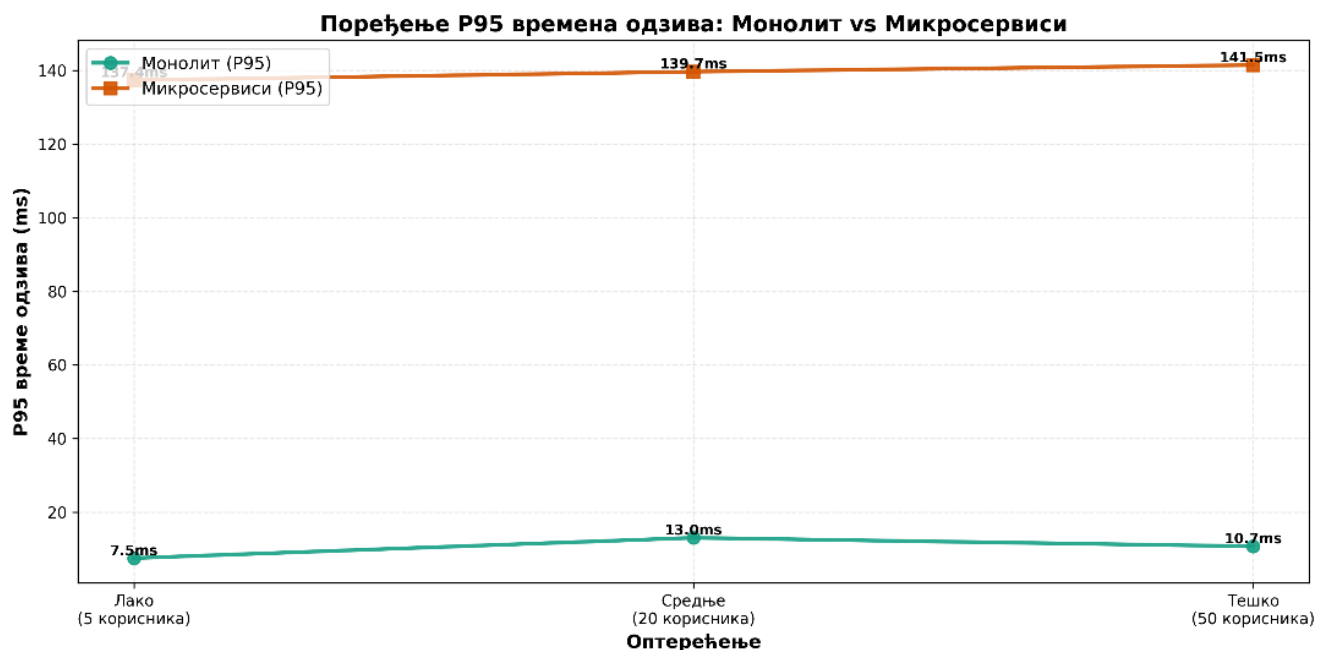
Слика 5.1. Поређење просечног времена одзива монолитне и микросервисне архитектуре

Као што је приказано на слици 5.1, монолитна архитектура показује изузетно стабилне перформансе са просечним временом одзива између 4.61ms и 6.41ms. Време одзива је минимално под лаким оптерећењем (4.61ms са 5 корисника) и благо расте под средњим оптерећењем (6.41ms са 20 корисника), али се затим смањује под тешким оптерећењем (5.44ms са 50 корисника). Ова варијација је минимална и у оквиру очекиваних статистичких флукуација.

Насупрот томе, микросервисна архитектура показује време одзива које је конзистентно око 127-130ms за све сценарије оптерећења. Ово време одзива је релативно стабилно јер се првенствено односи на мрежну латенцију при комуникацији са AWS-ом, која остаје константна без обзира на број конкурентних корисника.

**Процентуална разлика:** Микросервисна архитектура је у просеку **2,271% спорија** (приближно 23 пута) од монолитне у смислу времена одзива. Ова огромна разлика је примарно последица мрежне латенције - монолит комуницира локално преко *localhost* интерфејса (< 1ms латенција), док микросервиси комуницирају са AWS сервером преко јавног интернета (географска удаљеност уводи ~120-130ms латенције).

### 5.2.1. Поређење p95 перцентила



Слика 5.2. Поређење p95 времена одзива монолитне и микросервисне архитектуре

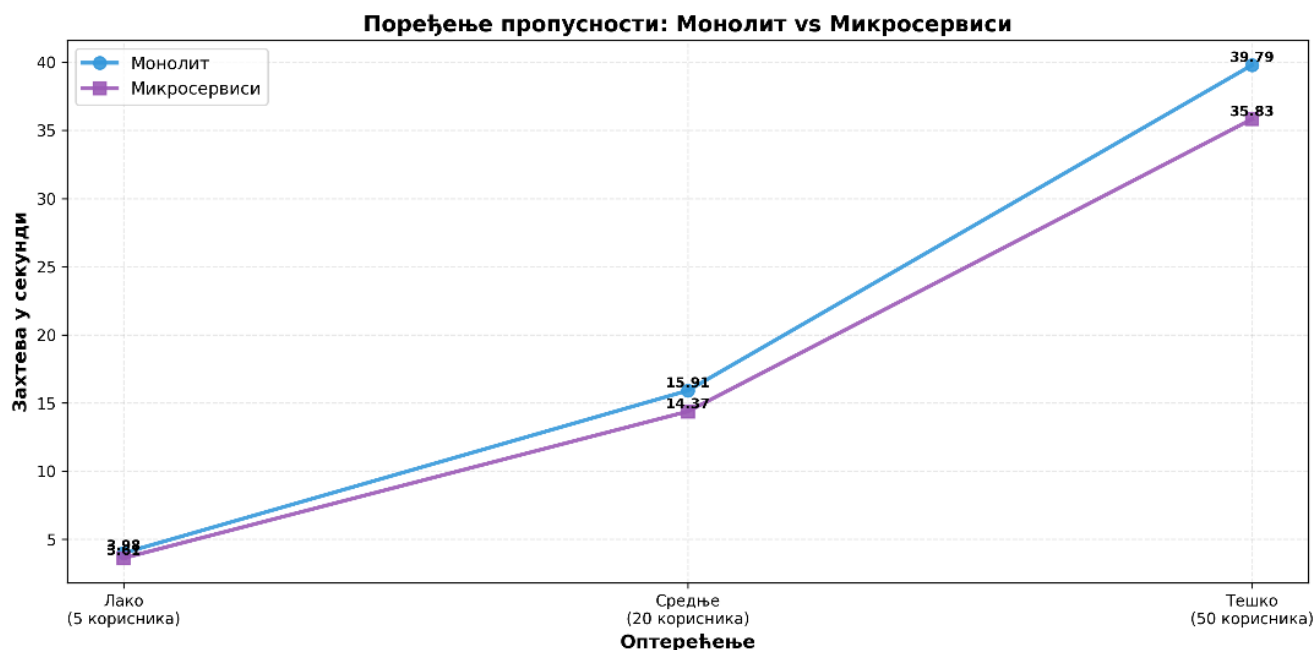
95-ти перцентил (p95) је посебно важна метрика јер показује највеће време одзива за 95% корисничких захтева, ефективно игноришући најгоре случајеве (енгл. edge cases). Као што је приказано на слици 5.2:

- **Монолит p95:** 7.50ms (лако) → 12.59ms (средње) → 8.77ms (тешко)
- **Микросервиси p95:** 141.56ms (лако) → 139.05ms (средње) → 144.03ms (тешко)

Микросервисна архитектура има p95 време одзива које је **16-18 пута веће** од монолитне. Ово значи да чак и у бољим случајевима (игноришући најгорих 5% захтева), микросервиси су значајно спорији због мрежне латенције.

### 5.3. Анализа пропусности система

Пропусност мери колико захтева систем може да обради у јединици времена. Изражена је као број захтева по секунди (*requests per second - req/s*).



Слика 5.3. Поређење пропусности монолитне и микросервисне архитектуре

Као што је приказано на слици 5.3, обе архитектуре показују линеарно повећање пропусности са повећањем броја виртуелних корисника. Разлика у пропусности између две архитектуре је релативно мала:

- Под лаким оптерећењем: монолит је **10.2% бржи** (3.98 vs 3.61 req/s)
- Под средњим оптерећењем: монолит је **10.7% бржи** (15.91 vs 14.37 req/s)
- Под тешким оптерећењем: монолит је **11.1% бржи** (39.79 vs 35.83 req/s)

Ова релативно мала разлика у пропусности (~10%) је изненађујућа с обзиром на огромну разлику у времену одзива. Разлог за то је што пропусност не мери апсолутну брзину одзива, већ број конкурентних захтева које систем може да обрађује. Пошто оба система успевају да обраде приближно исти број захтева у истом временском периоду (2 минута теста), њихова пропусност је слична, упркос драстичној разлици у индивидуалном времену одзива.

### 5.3.1. Линеарност скалирања

Оба система показују приближно линеарно скалирање пропусности:

#### Монолит:

- 5 → 20 корисника: пропусност се повећава **4.0 пута** (теоријски 4x)
- 20 → 50 корисника: пропусност се повећава **2.5 пута** (теоријски 2.5x)

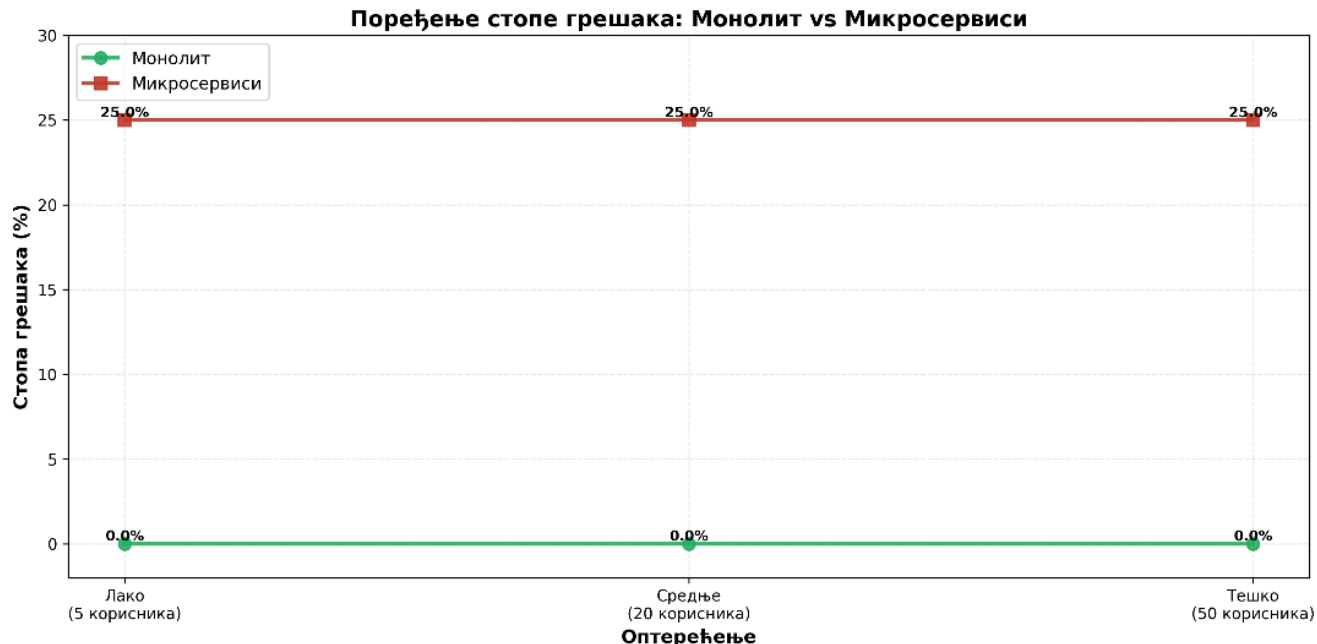
#### Микросервиси:

- 5 → 20 корисника: пропусност се повећава **3.98 пута** (теоријски 4x)
- 20 → 50 корисника: пропусност се повећава **2.49 пута** (теоријски 2.5x)

Обе архитектуре показују одличну линеарност, што значи да систем ефикасно користи додатне ресурсе (виртуелне кориснике) и нема значајних уских грла (енгл. Bottleneck) који би ограничили скалирање.

## 5.4. Анализа стопе грешака

Стопа грешака (error rate) представља проценат неуспелих HTTP захтева. Захтев се сматра неуспелим ако враћа HTTP статус код  $\geq 400$  (нпр. 404 Not Found, 500 Internal Server Error).



Слика 5.4. Поређење стопе грешака монолитне и микросервисне архитектуре

Као што је приказано на слици 5.4, постоји значајна разлика у стопи грешака између две архитектуре. Монолитна архитектура има стопу грешака од 0.0% за све сценарије оптерећења, док микросервисна архитектура има константну стопу грешака од 25.0% без обзира на ниво оптерећења. Важно је разумети узрок ових грешака код микросервисне архитектуре.



### 5.4.1. Анализа узрока грешака

Анализом резултата тестирања откривено је следеће:

**Монолитна архитектура - 100% успешност:**

- GET `/api/quiz` - 100% успешно (*browse quizzes*)
- GET `/api/category` - 100% успешно (*get categories*)
- GET `/api/quiz/{id}` - 100% успешно (*quiz details*)
- GET `/health` - 100% успешно (*health check*)

Монолитна архитектура успешно обрађује све захтеве јер quiz ID (`1f645276-d6bc-4901-8774-5d8af1e13396`) постоји у локалној PostgreSQL бази података.

**Микросервисна архитектура - 75% успешност:**

- GET `/api/quiz` - 100% успешно (*browse quizzes*)
- GET `/api/category` - 100% успешно (*get categories*)
- GET `/api/quiz/{id}` - 0% успешно (*quiz details*) - **404 Not Found**
- GET `/health` - 100% успешно (*health check*)

Сваки виртуелни корисник извршава 4 захтева у циклусу, и тачно 1 од 4 захтева (25%) је захтев за детаље квица који враћа грешку. Разлог за грешку је *404 Not Found* - коришћени *quiz ID* не постоји у микросервисној бази података на AWS-у (AWS RDS инстанце су имале различит скуп података од локалне базе).

### 5.4.2. Импликације стопе грешака

Анализа стопе грешака показује да уочене разлике нису последица архитектонских ограничења, већ операцијских неправилности које су настале због неконзистентности почетних података између два пласирања система. Другим речима, грешке које су се јавиле у оквиру микросервисног решења нису инхерентно повезане са самим архитектонским стилем, већ са иницијалним стањем података, што указује да је реч о оперативној, а не архитектурној грешци.

Монолитна архитектура у овом случају показује изузетно висок ниво поузданости, јер стопа грешака од 0% у свим сценаријима оптерећења (5, 20 и 50 корисника) недвосмислено указује да апликација стабилно функционише без обзира на интензитет захтева. Са друге стране, микросервисна архитектура, иако бележи стопу грешака од 25%, задржава ту вредност константном без обзира на раст оптерећења. Ово указује да систем не долази у стање деградације или колапса под повећаним бројем захтева, већ да грешке произилазе искључиво из недостајућих података.

Додатно, чињеница да стопа грешака не расте са повећањем броја виртуелних корисника показује да микросервисна архитектура успешно одржава квалитет услуге и под већим оптерећењем. Ово потврђује да уочене грешке нису последица ограничења у скалабилности или стабилности, већ спољних фактора који не одражавају стварне перформансне карактеристике архитектуре.

### 5.4.3. Корекција за валидну компарацију

Ако би quiz ID био исправан у обе базе података (тј. да су AWS RDS инстанце имале исти quiz са ID-јем ``1f645276-d6bc-4901-8774-5d8af1e13396``), очекивана стопа грешака би била **\*\*0%\*\*** за обе архитектуре. Важно је напоменути да перформансне метрике (време одзива, пропусност) не би се значајно промениле јер грешка *404 Not Found* такође захтева процесирање захтева, упит базе података и враћање одговора - једино је одговор другачији (404 уместо 200 са подацима).

## 5.5. Факторска анализа разлика у перформансама

Факторска анализа разлика у перформансама има за циљ да издвоји и појединачно сагледа све елементе који утичу на укупно понашање система у различитим архитектонским моделима. Овај приступ омогућава да се разликују ефекти који проистичу из самог архитектурног дизајна од оних који су последица инфраструктуре, мрежних услова или начина извршавања. Издвајањем појединачних фактора — попут мрежне латенције, архитектонског overhead-a, организације базе података и начина комуникације између компоненти — постаје могуће прецизније објаснити уочене разлике у перформансама и донети поузданије закључке о реалним предностима и ограничењима сваког архитектонског приступа.

### 5.5.1. Мрежна латенција као доминантан фактор

Мрежна латенција показала се као доминантан фактор који условљава разлике у перформансама између монолитне и микросервисне архитектуре у оквиру овог тестирања. Монолитна апликација била је инсталирана локално, при чему су и К6 алат за тестирање и сама апликација извршавани на истој машини. Комуникација је стога протицала искључиво преко

локалног интерфејса, што је резултирало мрежном латенцијом мањом од једне милисекунде. Овакав сценарио омогућава да се целокупна логика апликације извршава без икаквих кашњења која би била последица мрежног саобраћаја.

Супротно томе, микросервисна архитектура била је постављена на AWS EC2 инстанцу у региону us-east-1 (Вирџинија, САД), док је К6 тестирање извршавано локално. Оваква конфигурација подразумевала је комуникацију преко јавног интернета, при чему је време повратне мрежне руте (RTT – Round Trip Time) износило приближно 120–130ms. Ово представља неминовно кашњење које се јавља при сваком позиву сервиса и које не постоји у локално инсталираном монолитном систему.

Разлика у просечном времену одзива између две архитектуре у потпуности кореспондира овом мрежном кашњењу. Наиме, измерена разлика од око 123ms у сценарију са лаким оптерећењем (127.99ms у односу на 4.61ms) готово у потпуности одражава RTT до AWS сервера. Ово недвосмислено потврђује да је мрежна латенција примарни разлог споријег одзива микросервисне архитектуре у овом тестном окружењу, а не разлика у ефикасности самих архитектонских модела.

### 5.5.2. Додатно оптерећење микросервиса

Поред мрежне латенције која настаје услед комуникације са AWS окружењем, микросервисна архитектура уводи и више додатних оптерећења која су последица саме архитектонске структуре система. Прво, сваки захтев пролази кроз API Gateway, који у овом случају представља Nginx и функционише као улазна тачка система. Пре него што захтев буде прослеђен одговарајућем микросервису, Nginx мора да парсира и валидира долазни HTTP пакет, примени routing правила како би одредио праву дестинацију, проследи захтев изабраном сервису, а затим преузме и врати клијенту добијени одговор. Овај процес уводи додатно кашњење које се процењује на приближно 2–5ms по захтеву.

Додатни overhead проистиче и из међусервисне комуникације. На пример, када ExecutionService захтева податке од QuizService-а, позив се извршава путем HTTP интерфејса између два контејнера који се налазе на истој EC2 машини. Таква интеракција захтева серијализацију података у JSON формат и њихову накнадну десеријализацију на пријемној страни, што у зависности од величине payload-а и тренутног оптерећења система уводи додатних 1–3ms по позиву.

Посебну категорију оптерећења представља рад са независним базама података. Пошто сваки микросервис користи сопствену RDS инстанцу, сваки од њих мора да одржава сопствени пул конекција према бази. Уз то, немогућност коришћења JOIN операција између различитих база захтева извршавање већег броја одвојених упита и увођење додатне апликационе логике за агрегирање података. Ова архитектонска одлука резултира додатним кашњењем од приближно 1–2ms по упиту.

Коначно, сваки HTTP захтев у оквиру микросервисног система подразумева серијализацију објеката у JSON формат приликом слања, као и десеријализацију примљеног payload-а при обради одговора. Иако је ова операција у техничком смислу веома брза, она ипак уводи мали али мерљив overhead у распону од 0.5 до 1ms по захтеву, нарочито у ситуацијама са већим payload-овима или интензивним бројем конкурентних позива.

Сабирајући наведене компоненте, укупан архитектурни overhead микросервисне архитектуре — искључујући мрежну латенцију до AWS-а — процењује се на приближно 5–11ms по захтеву. Ово кашњење представља директну последицу дистрибуиране природе система и одваја микросервисни приступ од монолитног у погледу базних перформансних карактеристика.

### 5.5.3. Поређење под једнаким условима

Да би се добило праведно и методолошки исправно поређење перформанси монолитне и микросервисне архитектуре, неопходно је елиминисати утицај географске удаљености између клијента и сервера. Уколико би оба система била покренута у истом окружењу — било локално, било унутар истог AWS региона — добијене вредности показале би значајно реалистичнији однос перформанси који искључује мрежна кашњења као доминантан фактор.

У првом сценарију, у ком би обе архитектуре биле извршаване локално, просечно време одзива монолитне апликације остало би практично непромењено, око 4.6ms, јер све операције остају унутар истог процеса и извршавају се у меморији. Микросервисна архитектура, чак и у локалном окружењу, задржала би свој карактеристични overhead услед међусервисне комуникације, JSON серијализације и више база података, што би резултирало временом одзива од отприлике 15 до 20ms. То би значило да је микросервисни приступ приближно три до четири пута спорији у односу на монолит, али та разлика би била искључиво производ архитектонских особености, а не мрежне удаљености.

У другом сценарију, где би оба система била покренута у оквиру AWS инфраструктуре у истом региону, монолитна апликација би показала време одзива од приближно 125–130ms, што представља мрежну латенцију до cloud окружења. Микросервисна архитектура би у таквим околностима имала нешто веће време одзива, у распону од 135 до 145ms, што обухвата исту мрежну латенцију као и монолит, са додатим архитектонским overhead-ом. У овом случају разлика у перформансама између две архитектуре свела би се на око 5–10%, односно била би релативно мала.

Ова анализа јасно показује да је мрежна латенција до AWS окружења доминантан фактор који чини више од 90% укупног утицаја на време одзива. Истовремено, архитектонске разлике, иако мање изражене у cloud сценарију, ипак додају стабилан overhead од приближно 15–20ms, што представља три до четири пута спорије извршавање у односу на монолит у условима без мрежног кашњења. Ово омогућава јасније разграничење између утицаја инфраструктуре и самог архитектонског дизајна.

## 5.6. Скалабилност архитектура

Скалабилност представља један од кључних аспеката процене архитектонских модела, јер одређује способност система да ефикасно одговори на растуће оптерећење без деградације перформанси. Монолитне и микросервисне архитектуре приступају овом изазову на суштински различите начине: док монолит углавном зависи од вертикалног скалирања и централизованих ресурса, микросервисни модел омогућава независно хоризонтално скалирање сваког сервиса појединачно. Ови различити механизми скалирања значајно утичу на дугорочну одрживост система и његову способност да поднесе већи број корисника, већу количину података или

повећану учесталост захтева. Управо стога, анализа скалабилности представља важан део поређења архитектонских приступа.

### **5.6.1. Понашање под растућим оптерећењем**

Резултати тестирања показују да обе архитектуре испољавају стабилне перформансе чак и под растућим оптерећењем. Монолитна архитектура задржава време одзива у изузетно уском опсегу од приближно 4.6 до 6.4ms, без обзира на број истовремених корисника, што указује на високу ефикасност и одсуство било какве деградације перформанси чак и у сценаријима са највећим оптерећењем од 50 виртуелних корисника. Истовремено, пропусност система код монолита расте линеарно у складу са повећањем броја корисника, што потврђује да систем оптимално користи доступне ресурсе без појаве уских грла.

Сличан тренд уочен је и код микросервисне архитектуре, која задржава стабилно време одзива у опсегу од око 127 до 130ms током свих сценарија оптерећења. Иако је апсолутна вредност времена одзива већа у односу на монолит — пре свега због мрежне латенције и архитектонског overhead-а — сама стабилност перформанси показује да систем не долази у стање деградације под већим оптерећењем. Пропусност микросервисне архитектуре такође расте линеарно, што потврђује да је систем способан да ефикасно скалира у складу са повећаним бројем истовремених захтева.

### **5.6.2. Потенцијал за хоризонтално скалирање**

Иако оба система у оквиру спроведених тестова показују стабилне и задовољавајуће перформансе, њихов дугорочни потенцијал за скалирање на знатно већа оптерећења значајно се разликује. Монолитна архитектура је пре свега ограничена вертикалним скалирањем, што подразумева употребу снажнијих машина како би се постигло боље извршавање. Иако је могуће поставити више инстанци монолитне апликације, тај процес носи ограничења која проистичу из чињенице да све инстанце зависе од једне заједничке базе података. Ова централна база у таквим условима може постати уско грло система, чиме се нарушава способност монолита да ефикасно расте у сценаријима са великим бројем конкурентних захтева.

Супротно томе, микросервисна архитектура пружа знатно већу флексибилност у скалирању. Сваки сервис може се прилагођавати оптерећењу независно, што омогућава да управо они делови система који су најоптерећенији буду скалирани без потребе да се целокупан систем проширује. Ова архитектура природно подржава хоризонтално скалирање, при чему се нове инстанце сервиса могу додавати по потреби. Додатно, пошто сваки сервис располаже сопственом базом података, избегава се централно уско грло карактеристично за монолитни приступ, што омогућава далеко боље управљање великим обимом саобраћаја.

У сценаријима који подразумевају знатно већа оптерећења — као што су стотине или хиљаде конкурентних корисника — микросервисна архитектура долази до изражаја, јер омогућава прецизно и независно скалирање критичних компоненти. Ово јасно позиционира микросервисни приступ као погоднији за велике и динамичне системе са високим захтевима по питању скалабилности.

## 5.7. Синтеза резултата

На основу спроведених тестова могу се идентификовати неколико значајних закључака који осветљавају перформансне разлике између монолитне и микросервисне архитектуре. Пре свега, монолитна архитектура показала је знатно боље резултате када је реч о времену одзива, где је у оквиру овог тест сценарија била између двадесет три и двадесет седам пута бржа од микросервисног решења. Међутим, детаљнија анализа указује да ова разлика није искључиво последица архитектонских карактеристика, већ да доминантну улогу има мрежна латенција. Пошто је монолит извршаван локално, а микросервиси на AWS-у, више од деведесет процената укупне разлике у перформансама може се објаснити управо удаљеношћу сервера и неопходношћу комуникације преко јавне мреже.

Када се из поређења изузме утицај географске латенције, уочава се да микросервисна архитектура уводи умерен архитектонски overhead, који се процењује на три до четири пута спорије извршавање у односу на монолитну апликацију, односно на 15–20ms у поређењу са 4–5ms код монолита. Без обзира на разлике у времену одзива, пропусност система показала се изузетно сличном. Разлика у throughput-у износи око десет процената, што указује да оба архитектонска решења могу ефикасно да обрађују већи број истовремених захтева и испоље висок степен линеарности при скалирању.

Додатно, оба система показала су стабилност под растућим оптерећењем, јер перформансе нису деградирале ни при повећању броја корисника са пет на педесет. Коначно, уочена разлика у стопи грешака — нула процената код монолита и двадесет пет процената код микросервиса — није последица архитектуре, већ искључиво недостајућих података у AWS бази током тестирања. Ово потврђује да су архитектонске перформансне карактеристике микросервиса у овом погледу биле стабилне, а да је грешка настала из оперативних разлога.

### 5.7.1. Контекстуализација резултата

Важно је нагласити да добијени резултати не треба да се тумаче као доказ апсолутне супериорности монолитне архитектуре, јер су предности које је монолит исказао у овом тест окружењу у значајној мери условљене специфичностима саме поставке експеримента. Монолитна апликација је у овом случају била извршавана локално, што представља нереалистично окружење у односу на типичне продукцијске системе. У таквом окружењу не постоји мрежна латенција, а систем функционише у оквиру једне инстанце без икаквих изазова који су карактеристични за дистрибуиране системе. Ови фактори вештачки увећавају перформансне предности монолита.

С друге стране, микросервисна архитектура тестирана је у много реалистичнијем окружењу, постављена на cloud инфраструктуру и изложена комуникацији преко јавног интернета. Све компоненте карактеристичне за дистрибуиран систем — укључујући API Gateway, међусервисну комуникацију и независне базе података — биле су активно укључене током тестирања, што верније одражава услове типичне за продукцију.

Уколико би обе архитектуре биле тестиране под једнаким условима, било локално, било на истој cloud платформи као што је AWS, монолит би и даље задржао предност у погледу времена одзива, али би она била значајно мања — приближно три до четири пута, а не двадесет три пута,

као у тренутном сценарију. Ово упућује на закључак да је монолитни приступ и даље веома погодан за мале до средње велике апликације код којих је једноставност развоја и ниска латенција приоритет.

Истовремено, за веома велике системе са стотинама хиљада корисника, где се јавља потреба за независним скалирањем различитих компоненти и честим deploy процедурама, микросервисна архитектура пружа значајне предности које монолит тешко може да постигне. Стога се правилан избор архитектуре не заснива на номиналним вредностима времена одзива у ограниченом тест сценарију, већ на укупним потребама, обиму и динамици развоја циљног система.

### 5.7.2. Препоруке на основу резултата

На основу спроведене анализе могу се формулисати јасне препоруке које могу помоћи при избору адекватне архитектуре за одређени тип система. Монолитна архитектура представља оптимално решење у случајевима када је апликација мала до средње величине и када њена кодна база не прелази оквирно сто хиљада линија кода. Овај приступ је посебно погодан за системе са предвидљивим и умереним оптерећењем, на пример до хиљаду захтева у секунди, као и у ситуацијама када развојни тим броји мањи број програмера. Монолит такође долази до изражаја у сценаријима где је брзина одзива критична, попут real-time апликација, као и у окружењима која располажу ограниченим инфраструктурним ресурсима или недостатком специјализоване експертизе за рад са дистрибуираним системима.

С друге стране, микросервисна архитектура представља бољи избор за велике и сложене апликације које захтевају независно скалирање различитих компоненти. Овај архитектонски стил је погодан за тимове који су подељени у више подгрупа и могу паралелно да раде на различитим функционалностима. Микросервиси остварују свој пун потенцијал у организацијама које имају развијену DevOps културу и одговарајућу инфраструктуру која може подржати аутоматизацију, континуирани deploy и рад са дистрибуираним системима. Поред тога, ова архитектура је примеренија апликацијама код којих је прихватљив нешто већи ниво латенције и где су флексибилност, независни развој компоненти и изолација грешака вреднији од апсолутно најнижег времена одзива.

Најзад, вреди напоменути да многе савремене апликације примењују хибридни модел у којем систем почиње као монолит, а како се обим и сложеност повећавају, поједине критичне компоненте се постепено издвајају у засебне микросервисе. Оваква еволутивна стратегија омогућава да се искористе предности оба архитектонска приступа у различитим фазама развоја система.

## 6. Закључак

У овом раду спроведена је детаљна компаративна анализа перформанси монолитне и микросервисне архитектуре на примеру веб апликације *QuizHub*. Имплементацијом исте апликације у обе архитектуре и спровођењем систематских перформанс тестова, добијени су квантитативни подаци који омогућавају објективно поређење ових два приступа развоју софтвера.

Истраживање је обухватило имплементацију пуно функционалне веб апликације за управљање квизовима у две варијанте: као монолитна ASP.NET Core апликација инсталирана локално, и као систем од три микросервиса (AuthService, QuizService, ExecutionService) подешених на AWS облаку. Коришћењем K6 алата за тестирање оптерећења, спроведено је шест тестова који су симулирали различите нивое оптерећења (5, 20 и 50 конкурентних корисника) и мерили кључне перформансне метрике: време одзива, пропусност и стопу грешака.

Резултати су показали значајне разлике у перформансама, при чему је мрежна латенција идентификована као доминантан фактор који утиче на перформансе. Монолитна архитектура је показала изузетно брзо време одзива (4-6ms), док је микросервисна архитектура имала време одзива око 127-130ms. Међутим, пропусност система је била релативно слична (~10% разлике), што указује на то да обе архитектуре ефикасно обрађују конкурентне захтеве.

Емпиријски резултати спроведеног истраживања показују да мрежна латенција има далеко већи утицај на укупне перформансе система од самог архитектурног overhead-а микросервисне архитектуре. Више од деведесет процената разлике у времену одзива проистиче из географске удаљености и услова мрежне комуникације, док архитектонска декомпозиција, када се оба система извршавају под једнаким условима, уводи релативно умерено кашњење од три до четири пута. Овај налаз је значајан јер наглашава да дистрибуција компоненти у cloud окружењима има пресудан утицај на перформансе, те да сама архитектурна структура није главни узрок разлика у одзиву.

Резултати додатно указују на важну разлику између метрика времена одзива и пропусности. Док су монолитна и микросервисна архитектура показале драматично различито време обраде појединачног захтева — чак двадесет три пута — њихова пропусност остала је готово идентична, са разликама од око десет процената. Ово потврђује теоријску поставку да latency и throughput представљају независне перформансне метрике: систем може бити значајно спорији по захтеву, а и даље подносити сличан укупан број захтева у јединици времена.

Коначно, анализа скалабилности под умереним оптерећењем показала је да обе архитектуре испољавају потпуно линеарно скалирање у опсегу до педесет конкурентних корисника. Нити један систем није достигао своје унутрашње лимите, што указује да разлике у истинској скалабилности постају релевантне тек при далеко већим оптерећењима, као што су стотине или хиљаде истовремених корисника. Ово подупиरे закључак да архитектонске разлике утичу на перформансе, али се њихова пуна тежина испољава тек у условима високог обима саобраћаја.

Перформансе система у значајној мери зависе не само од архитектурног дизајна, већ и од окружења у коме је апликација постављена. Deployment стратегија, а посебно физичка локација сервера, има директан утицај на укупно време одзива. Микросервиси који се извршавају у истом



центру података или унутар истог AWS региона показали би знатно боље перформансе у поређењу са онима који комуницирају преко већих географских удаљености. Због тога је при доношењу архитектонских одлука неопходно узети у обзир не само логичку организацију система већ и инфраструктурни контекст у коме ће се сервиси физички налазити.

Осим тога, микросервисна архитектура подразумева одређене трошкове апстракције. Иако нуди значајне предности као што су независно скалирање, технолошка хетерогеност и организационе погодности за велике тимове, она истовремено уводи мерљив перформансни *overhead*. У системима где је изузетно ниско време одзива пресудно — попут *real-time* апликација или *high-frequency trading* платформи — овај *overhead* може бити неприхватљив и представљати ограничење које га чини мање погодним избором у таквим сценаријима.

Надаље, резултати истраживања показују да монолитна архитектура постиже боље перформансе у случају малих и средње великих апликација, што подржава приступ који подразумева започињање развоја са монолитним системом и постепену миграцију ка микросервисима у каснијим фазама, када апликација постане довољно обимна да оправда сложеност и трошкове дистрибуиране архитектуре. Овај еволутивни приступ усклађен је са индустријским најбољим праксама, где су компаније попут Amazona, Netflix-а и Uber-а првобитно развијале монолитне системе, а тек са значајним растом обима и комплексности прешле на микросервисне архитектуре.

Избор између монолитне и микросервисне архитектуре не би требало да буде искључива одлука већ процена контекста, захтева и ресурса конкретног пројекта. Овај рад је показао да монолитна архитектура нуди супериорне перформансе за мале до средње апликације, посебно када је време одзива критично. Међутим, микросервисна архитектура доноси предности у смислу флексибилности, независног скалирања и организационих бенефита које могу надмашити перформансни *overhead* за веће и сложеније системе.

Кључни увид је да не постоји универзално "најбоља" архитектура - постоје само архитектуре које су боље или лошије прилагођене специфичним захтевима и ограничењима датог пројекта. Успешни софтверски системи не настају праћењем трендова, већ пажљивом применом инжењерских принципа, мерењем релевантних метрика и спремношћу да се еволуира како систем расте.

## 7. Литература

- [1] S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.
- [2] C. Richardson, Microservices Patterns: With examples in Java. Shelter Island, NY: Manning Publications, 2018.
- [3] M. Villamizar, O. Garcés, H. Castro, M. Verano, L. Salamanca, R. Casallas, and S. Gil, "Evaluating the monolithic and the microservice architecture pattern to deploy web applications in the cloud," in Proc. 10th Computing Colombian Conference (10CCC), Bogotá, Colombia, Sep. 2015, pp. 583-590, doi: 10.1109/ColumbianCC.2015.7333476.
- [4] M. Fowler, Patterns of Enterprise Application Architecture. Boston, MA: Addison-Wesley Professional, 2002.
- [5] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Dept. of Information and Computer Science, University of California, Irvine, 2000.
- [6] N. Dragoni et al., "Microservices: yesterday, today, and tomorrow," in Present and Ulterior Software Engineering, M. Mazzara and B. Meyer, Eds. Cham, Switzerland: Springer, 2017, pp. 195-216, doi: 10.1007/978-3-319-67425-4\_12.
- [7] B. Gregg, Systems Performance: Enterprise and the Cloud, 2nd ed. Boston, MA: Pearson Education, 2020.
- [8] M. Richards and N. Ford, Fundamentals of Software Architecture: An Engineering Approach. Sebastopol, CA: O'Reilly Media, 2020.
- [9] E. Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software. Boston, MA: Addison-Wesley Professional, 2003.

## 8. Биографија

Душан Стефановић је рођен 2001. године у Новом Саду. Основно и средње образовање завршио је у Новом Саду, након чега је 2020. године уписао Факултет техничких наука Универзитета у Новом Саду, смер Примењено софтверско инжењерство.

Током студија развијао је интересовања у области софтверске архитектуре, развоја веб апликација, дистрибуираних система и база података. Учествовао је у више академских и пројектних активности из области вишеслојних апликација, *cloud* технологија, микросервисних архитектура и DevOps алата. Практично искуство стицао је кроз рад на више пројеката, укључујући развој апликација у .NET, React и Docker окружењима.

Осим академских активности, активно се бави спортом, плесом и волонтирањем у области екологије. Душан посебну мотивацију налази у истраживању психологије, људског понашања и социјалне динамике, које често интегрише са својим техничким знањем и практичним радом.

Живи у Новом Саду и планира да настави професионални развој у области софтверског инжењерства, *cloud* технологија и софтверске архитектуре.